

7. 基本圖論演算法

「BFS、DFS、與 Dijkstra 演算法之間有何差異？」

「BFS 像是小寶寶自己找父母，出生第一眼就到的就認做 parent；DFS 像是幫孩子找爹娘，最後結婚生下的才是你孩子的 parent；Dijkstra 則是長大後自己找乾爹乾媽，最有利的才是乾爹乾媽。」

「那麼，Prim 與 Dijkstra 演算法的差異呢？」

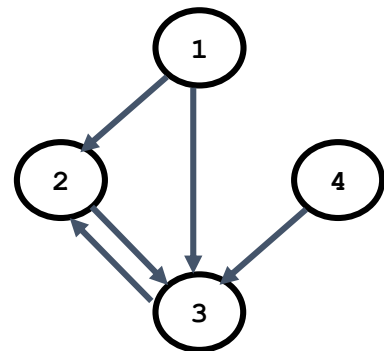
「認乾爹乾媽的依據稍有不同。Prim 只看乾爹乾媽對你有多好，Dijkstra 則除了對你多好之外，還有他從祖先繼承了多少會留給你的財產。」

這一章介紹圖 (graph)，圖是用來表達二元關係的一種方式，有很多重要的應用，也發展出很多特殊的演算法，這些演算法都是過去科學家研究發展出來的，也就是說，我們要了解它的話，需要靠學習與理解，很難只靠基本的演算法原則自己去想出來。圖在競賽程式中也扮演了很重要的腳色，但基於高中資訊領域的範圍與 APCS 的考試範圍，這份教材只會包含一些圖的基本演算法。本章內容包含：圖的基本觀念與名詞、儲存圖的資料結構、廣度優先搜尋、深度優先搜尋、以及 DAG 與拓撲順序，在本章最後我們則介紹了三個比較進階的技術，包括 Dijkstra 演算法、併查集、以及最小生成樹。至於在樹狀圖上做計算的相關問題則留在下一章。

7.1. 基本觀念與名詞

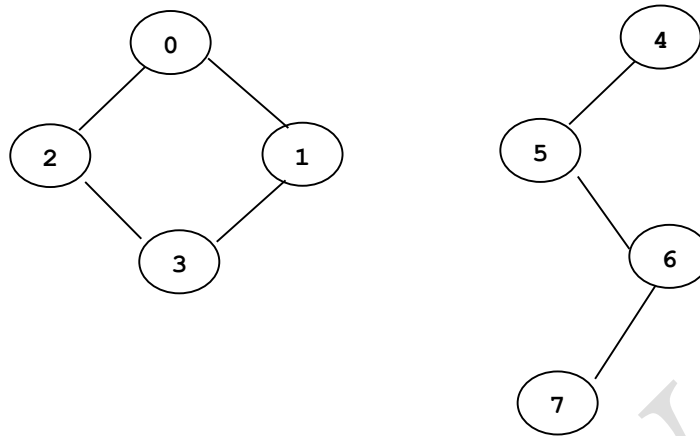
資料結構與演算法中稱的圖 (graph) 與我們日常生活中的圖 (地圖，圖形，藍圖) 不太一樣，這裡指的是圖論 (graph theory) 中的圖，而圖論算是離散數學中的一支。Graph 基本上是二元關係的一種表達方式，目的在研究一群事物中兩兩之間的關係。一個圖寫作 $G=(V,E)$ ，表示一個圖名字是 G ，由點的集合 V 與邊的集合 E 組成，其中 V 是點集合，也就是要研究的對象；而 E 是邊的集合，也就是誰與誰之間存在這種關係。

右圖是一個例子，圖中四個圓圈代表我們要研究的四個人，研究的關係是誰喜歡誰，因此，有一條線從 1 指到 2 表示 1 喜歡 2，記做 $(1,2)$ 。同樣的，圖中還有另外 4 個邊 $(1,3)$ 、 $(2,3)$ 、 $(3,2)$ 與 $(4,3)$ ，分別代表 1 喜歡 3、2 喜歡 3、3 喜歡 2 以及 4 喜歡 3。圖中沒出現的邊，就表示該二者之間沒有這種關係，例如 1 沒有喜歡 4，2 沒有喜歡 1。



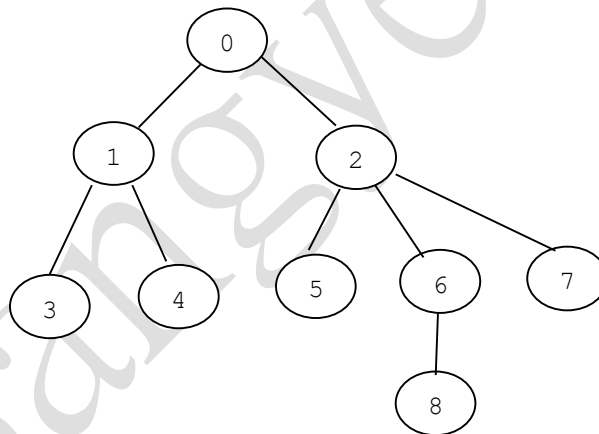
圖的術語很多，以下我們介紹一些基本術語，另外有一些在我們碰到的時候再介紹。

- 有向圖與無向圖(directed and undirected graph)：邊有方向性的圖是有向圖，通常以箭頭表示邊。有向圖表示的是非對稱的二元關係，如上圖的例子，1 喜歡 2 但 2 沒有喜歡 1，圖上有 $(1, 2)$ 但沒有 $(2, 1)$ ，有向圖上如果雙向關係都存在則需要用兩個邊表示，例如上圖中的 $(2, 3)$ 與 $(3, 2)$ 。無向圖的邊是沒有指明方向的，其實無向圖就是雙向圖，每一個邊都是雙向關係，無向圖表達的是一種對稱的關係，也就是對任意 a 與 b ， (a, b) 存在若且惟若 (b, a) 存在，或者說 (a, b) 與 (b, a) 是一樣的。
- 無加權圖與加權圖(unweighted and weighted graph)：圖上的點與邊都可以有權重，例如以點表示城市，邊代表道路的交通網圖，點的權重可能表示城市人口數，邊的權重可能表示路的長度。如果沒有說明，通常以 $G=(V, E, w)$ 表示一個邊上有權重的圖，其中 w 是一個邊的函數， $w(a, b)$ 代表邊 (a, b) 的權重。無加權圖就可以看成每個點(邊)的權重都是 1 的加權圖。
- 路徑與環路：從一個點 v_0 出發，經過某個邊 (v_0, v_1) 到達 v_1 ，再經由某個邊 (v_1, v_2) 到達 $v_2, \dots$ ，最後到達 v_k ，這樣的一個序列稱為路徑(path)，若任兩點之間最多只有一條邊(稱為 simple graph)，則我們可以用點的序列 $(v_0, v_1, v_2, \dots, v_k)$ 來代表這條路徑。一個路徑上如果沒有經過重複的點，就稱為簡單路徑(simple path)，若沒有特殊指明，通常路徑指的是簡單路徑。頭尾是相同的點的路徑則稱為環路(cycle/circuit)，也就是從某點出發經過若干點之後回到出發點。
在邊無加權的圖上，路徑的長度指的是經過的邊的數量，以上例來說，路徑長度是 k ；在邊有加權的狀況下，路徑的長度是經過邊的權重總和，這時候邊的權重往往也稱為長度。兩點之間可能有多條(或沒有)路徑，長度最短的路徑稱為最短路徑(shortest path)，最短路徑的長度通常稱為距離(distance)或強調稱為最短距離。在無加權圖中，每走一條邊可以比喻為走一步，最短距離就是最少走幾步可以到達目的點。
- 無向連通圖與連通區塊：一個無向圖如果任兩點之間都存在路徑相通，則稱為連通圖(connected graph)，否則稱為非連通圖(disconnected graph)。一個圖中一群彼此互相有路徑相通的點所形成的區塊稱為一個連通區塊(connected component)，但不被包含在其他連通區塊的才能稱為連通區塊。下圖是一個例子， $\{0, 1, 2, 3\}$ 形成一個連通區塊， $\{4, 5, 6, 7\}$ 是另外一個連通區塊。但 $\{0, 1, 2\}$ 不算。從圖形上看，連在一起的就是屬於一個連通區塊，以道路交通圖為例，則離島之間是不連通的，每個離島可能屬於單獨的連通區塊。



A disconnected graph with two connected components.

- 一個連通而無環路的圖稱為樹 (或樹狀圖)，如果是簡單圖，則樹的邊數必然等於點數減一。樹中如有一點被指定為根 (root)，則稱為有根樹，否則是無根樹。有根樹通常把根畫在最上方像一個家族圖，如下圖是一個有根樹，其中 0 是根。除了根以外，每個點的上方 (通往根的路徑上的第一個點) 一定恰有一點，這個點稱為他的



A rooted tree with 9 vertices and 8

parent，例如 4 與 3 的 parent 是 1；5, 6 與 7 的 parent 是 2；而 1 與 2 的 parent 是 0 (根)。若 a 是 b 的 parent，則稱 b 是 a 的孩子 (child)。只要指出每一個點的 parent 就可以惟一決定一棵樹。樹的相關問題很多，我們留在下一章介紹，但是這一章裡面也會碰到一些與樹有關的術語。

- 有向圖的強連通：在無向圖上，一個圖只要連在一起就是連通圖，任兩點之間皆有路徑相連。但在有向圖上比較複雜，因為有向圖的路徑必須依照箭頭方向前進，所以連在一塊的不一定有路徑可以到達。若對任意兩點 a 與 b ，圖上皆存在由 a 到 b 的路徑 (以及由 b 到 a 的路徑)，則稱為強連通圖 (strongly connected graph)。

- 點的分支度 (degree) 與鄰居：兩個點如有邊相鄰則稱為鄰居，無向圖上是互為鄰居，有向圖則要區分 in-neighbor 與 out-neighbor。鄰居的個數則為分支度 (degree)，也就是該點所接邊的個數 (假設是簡單圖)。同樣的，在有向圖上 degree 要分為 in-degree 與 out-degree。

一個圖 $G=(V,E)$ ，習慣上用 n 代表點數， m 代表邊數。圖的問題的輸入通常包含點與邊，所以輸入資料的大小是 $O(n+m)$ ，在表達演算法複雜度時，往往也需要表示成 n 與 m 的函數。我們知道邊數與點數是有一定的關係，在大部分狀況下 (simple graph)， m 不會超過 $O(n^2)$ ，所以有的時候也只表示成 n 的函數，但是表示成 n 與 m 的函數會更精準，特別是很多問題所要處理的圖的邊數 m 只在 $O(n)$ 的大小，這時把 m 列為複雜度參數就很重要了。

在分析圖的演算法時，常常涉及點與邊的關係，以下是一個簡單而常用的性質，也被稱為「握手定理」：

在無向圖中，各點的 degree 總和等於 $2m$ ；在有向圖中，各點的 in-degree 總和與 out-degree 總和，都等於 m 。

握手定理名稱的由來是這定理的無向圖版本可以描述成：在一場聚會中，每個人握手次數的總和是兩倍的握手發生次數。這定理很容易理解，因為一根邊會貢獻兩個 degree，只是不一定貢獻給哪個點。而在有向圖時，一根邊一定貢獻出一個 in-degree 與一個 out-degree。

7.2. 資料結構與基本演算法

在這一節中我們介紹處理圖的基本資料結構與幾個基本演算法。

7.2.1. 資料結構

要進行圖的資料計算，第一件事就是必須要能夠存取圖，也就是要有儲存圖的資料結構。常用的資料結構有兩種：鄰接矩陣 (adjacency matrix) 與鄰接串列 (adjacency list)。圖在處理過程中，最基本上的操作就是找出某個點的鄰居，使用不同的資料結構，程式就會不同的寫法。

一個圖 $G=(V, E)$ ，習慣上用 n 代表點數， m 代表邊數，而將點以 $0 \sim n-1$ 編號 (或是 $1 \sim n$)。所謂鄰接矩陣就是一個 $n \times n$ 的二維陣列 $A[i][j]$ ，其中 $A[i][j]=1$ 代表有一個邊 (i, j) ，若 $A[i][j]=0$ 則表示該邊不存在。在無向圖時，此矩陣必然是對稱矩陣，也就是對所有 i 與 j ， $A[i][j]=A[j][i]$ ；如果是有向圖，則不一定。至於對角線上 $A[i][i]$ 則要看應用而定，在某些問題中， $A[i][i]$ 代表自迴圈 (self loop) 的邊，也就是由 i 到 i 的邊。

0	1	1	0	0	0	0	0
1	0	0	1	0	0	0	0
1	0	0	1	0	0	0	0
0	1	1	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	0	1	0
0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	0

對於加權圖 $G=(V, E, w)$ ，我們可以將 $A[i][j]$ 存 (i, j) 邊上的權重 $w(i, j)$ ，若邊的權重是正的，可以用 0 來表示邊不存在，但更常見的時候是以無窮大來表示，至於要以多大數字表示無窮大則看題目需要。

用鄰接矩陣來存圖最大的好處是簡單方便，要找出點 v 的 out-neighbor 就直接掃描列，找出那些 $A[v][i]=1$ 的 i ；若是要找 in-neighbor 就掃描行，找出那些 $A[i][v]=1$ 的 i 。

鄰接矩陣最大的麻煩是空間與時間效率。一個 n 個點的簡單圖，鄰接矩陣所需要的空間是 $O(n^2)$ 。另一方面，邊的個數 m 上限是 $O(n^2)$ ，如果邊數達到 $O(n^2)$ 我們通常稱為 dense graph，邊數是 $O(n)$ 的稱為 sparse graph。如果要處理的是 dense graph，那使用鄰接矩陣也是剛好。但是許多應用要處理的是 sparse graph，也就是說，如果用鄰接矩陣來存，矩陣裡面大部分都是 0，這顯然很沒有效率，因為資訊的量只有 $O(n+m)=O(n)$ 。這就是另外一種資料結構「鄰接串列」的優勢所在。

鄰接串列的概念很簡單：每個點把它的鄰居當成一個串列來存起來。這樣在有向圖中，鄰居的總數就是邊數 m ，在無向圖中，鄰居的總數就是 $2m$ (因為每個邊的兩端互為鄰居)。實作上呢？很多資料結構的書都是教用指標來做鏈結串列，將鄰居放進此串列中，但是在程式競賽與考試中，我們不需要這麼麻煩，我們可以使用 C++ STL 中的 vector，因為 vector 是長度可變的陣列，所以每一個點用一個 vector 來存它的鄰居就行了。

以下是範例的程式，讀入點數 n 與邊數 m ，然後讀取 m 個邊。這個範例程式只是要示範如何儲存圖，我們只把每個點的鄰居輸出並且計算一下 degree。第一支程式是示範鄰接矩陣，這裡假設是有向圖：

```
//ch7 demo adjacency matrix
#include <bits/stdc++.h>
using namespace std;
#define N 20
int main() {
    int adj[N][N]={0};
```

```

int n, m;
scanf("%d%d", &n, &m);
for (int i=0; i<m; i++) {
    int from, to;
    scanf("%d%d", &from, &to);
    adj[from][to]=1;
}
printf("adjacency matrix:\n");
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++)
        printf("%d ", adj[i][j]);
    printf("\n");
}
printf("deg = ");
int deg[N]={0};
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++)
        deg[i] += adj[i][j];
    printf("%d ", deg[i]);
}
printf("\n");
return 0;
}

```

第二支是示範鄰接串列：

```

//ch7 demo adjacency list
#include <bits/stdc++.h>
using namespace std;
#define N 20
int main() {
    vector<int> adj[N]; // adj[i]: neighbors of vertex i
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i=0; i<m; i++) {
        int from, to;
        scanf("%d%d", &from, &to);
        adj[from].push_back(to);
    }
    printf("adjacency list:\n");
    for (int i=0; i<n; i++) {
        printf("%d: ", i);
        for (int v: adj[i])
            printf("%d ", v);
        printf("\n");
    }
    printf("deg = ");
    for (int i=0; i<n; i++)
        printf("%d ", (int)adj[i].size());
    printf("\n");
    return 0;
}

```

```
}
```

不使用 **vector** 的鄰接串列做法 (*)

如果不使用 STL 的 `vector` 可以存鄰接串列嗎？當然是可以，事實上也不至於困難到考試或比賽時寫不出來，但還是使用 `vector` 最簡單，所以以下的說明只提供參考，並不建議用這個方法。

不使用 STL 物件時，一個做法就是自己用 `malloc()` 分配記憶體。給每一個點宣告一個整數指標，算一算每個點有多少鄰居就給他多少記憶體，或者先給一部分空間，當資料輸入多到不夠存時，在重新 `malloc()` 記憶體並將資料複製到新的位置，要留意的是不要每次只多給一個記憶體，通常的做法是不夠的時候就給兩倍，這樣根據均攤分析，每一個資料被複製搬移的時間只需要 $O(1)$ ，事實上這麼做差不多就是寫一個 `vector` 的 `push_back()`。

另外一個做法是宣告一個結構(struct)的陣列，結構中含兩個欄位放兩端點，如果邊上有加權就在加一個欄位。把所有的邊全放到這個陣列之後，呼叫一個排序讓這些邊依照第一端點排序，目的是讓相同端點的邊放在一起。接著我們對每一個點 v ，計算以 v 為端點的邊在此陣列中的開始位置 `start[v]`，這就完成前置的準備工作了。當我們要找 v 的鄰居時，就可以用

```
for (i=start[v]; i<start[v+1]; i++)
```

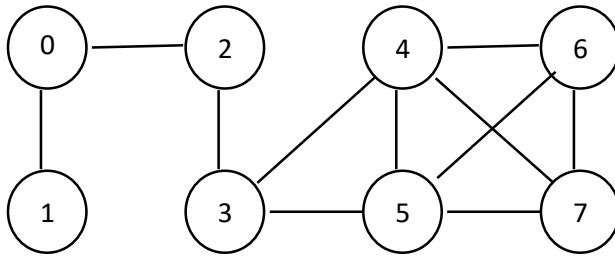
去找到 v 的鄰居。

這兩個方法只有在只能使用 C 而不能使用 C++ 的時機(環境)下，才建議這麼做，畢竟還是比較花費時間。當然，在某些狀況下，可能需要照著教科書用指標做鏈結串列，那就是另外一回事了。

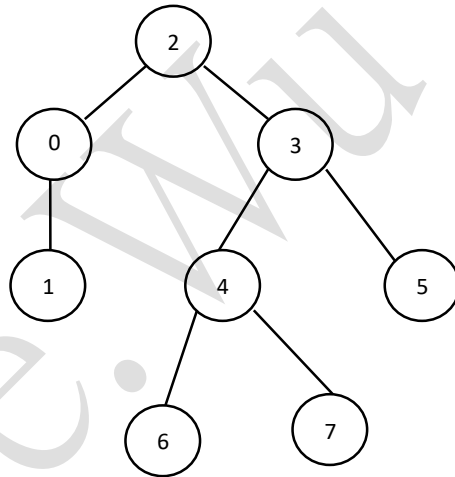
7.2.2. BFS

圖的搜尋是圖的基本問題：「輸入一圖 $G=(V,E)$ 以及其中一點 s ，要從 s 作為起點探索此圖，或者簡單的說，請問從 s 出發的話，可以到達哪些點。」基本有兩種常用的演算法，這兩種方法各有它們的特殊應用，在這一小節先介紹廣度優先搜尋 (Breadth-First Search, BFS)。(對了，英文要念得清楚些，不然很容易聽成麵包優先搜尋。)

在看 BFS 的演算法前，我們先以一個例子來說明算法要做的事情，以下是一個無向圖。



假設我們要以 $s=2$ 為起點以 BFS 探索這個圖。我們會先找出距離 s 一步的點，然後找兩步的點，...，一直到所有 s 能走到的點都找完為止。過程中我們將 s 比喻成一個家族的第 0 代祖先，與 s 距離 1 步的點就是第 1 代，距離 2 步的就是第 2 代，...。最後建出一個像類似家族世系圖的結果，如右圖。這個圖是一個樹狀圖，而且由 BFS 走出來的，我們稱它是一個 BFS tree。BFS tree 是一層一層的，每往下一層與起點 s 的距離就增加 1 步。每一個點會與上一層的某個點相連，這個點稱為它的 parent，在 BFS tree 中，每個點的 parent 就是第一個發現它的點。



在探訪第 i 代的某個點 u 時，例如 $u=4$ ，我們檢查 u 的所有鄰居 $\{3, 5, 6, 7\}$ ，這些鄰居有可能是第 $i-1$ 代 (如 3)、第 i 代 (如 5)、或是第 $i+1$ 代 (6 與 7)，所以我們必須加以區分，區分的方法是記錄每個點目前是否被發現過，當我們探訪完所有第 $i-1$ 代的點時，所有第 i 代與更早的點都一定已經被發現過，所以 u 的鄰居中未曾被發現的就是所要的第 $i+1$ 代的點。有了以上的認知，我們可以來說明 BFS 的演算法了，以下是 BFS 的演算法。對每一個點 v ，我們建立下列資料：

- $visit[v]$ 用來表示 v 點是否以被發現。
- $v.d$ 是距離起點 s 的距離。
- $v.p$ 是紀錄 v 的 parent，也就是發現它的點。

因為一個點可能會發現多個點，但我們必須依照發現的順序逐一探訪，所以我們準備一個 FIFO (先進先出) 的佇列 Q ，初始的時候只有起點 s 放入 Q 中，程式執行過程中，被發現的點就會被放入 Q 中，程式會執行到 Q 中沒有點為止。每一回合從 Q 中取出一點 u 來加以探訪，探訪就是檢查它的所有鄰居，其中尚未被發現的，就設定相關資料並放入 Q 中；如果是已經被發現的，就不予理會。


```

初始化對所有點  $v$ ,  $visit[v]=false$ ;
//  $v.d$  是  $s$  到  $v$  的距離,  $v.p$  是  $v$  在 BFS tree 上的 parent
BFS( $G, s$ ) {
    初始化一個 FIFO 的 queue  $Q$  只含  $s$ ;
     $s.d = 0$ ;  $visit[s] = true$ ;  $s.p = -1$ ;
    while ( $Q$  not empty) do
         $Q$  中移出一點  $u$ ;
        for each  $v \in adj[u]$  do
            if ( $visit[v] == false$ ) then
                 $visit[v] = true$ ;
                 $v.d = u.d + 1$ ;
                 $v.p = u$ ; // 第一個發現  $v$  的就是  $v$  的 parent
                將  $v$  加入  $Q$  中;
            end if
        end for
    end while
}

```

演算法其實並不困難，實作程式要搭配儲存圖的資料結構，因為每一個點最多只會進入 Q 中一次，時間複雜度不會超過 $O(n^2)$ 。如果以鄰接串列來儲存，複雜度是 $O(n+m)$ ，因為每個點從 Q 中出來時，做的事情是檢查它的所有鄰居，所以花的時間是它的鄰居數，也就是它的 degree，而根據握手定理，所有點的 degree 總和就是 $O(m)$ 。

BFS 通常有下列用途：

- 計算某個點出發可以到達的點。
- 在無加權圖中，計算一個點到其他點的距離，或是找兩點的最短路徑。
- 計算無向圖的 connected component。從一個點出發可以拜訪到該點所在的連通區塊的所有點，若對每一個點當起點進行 BFS，則可以找到所有連通區塊，但要注意應略過前面的點已經被拜訪過的點。

以下先看一個簡單例題，其他應用放在後面的例題與習題中。

P-7-1. 探索距離

輸入一個有向圖 G 與一個起點 s ，請計算由 s 出發可以到達的點數 (不包含 s)，並且計算這些可以到達的點與 s 的距離和，假設每個邊的長度均為 1。兩點之間可能有多個邊，邊的起點與終點未必不同。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 與 m ，代表圖的點數與邊數，圖的點是以 $0 \sim n-1$ 編號，第二行是 s 的編號，接下來有 m 行，每一行兩個整數 a 與 b 代表一個邊 (a, b) 。
 n 不超過 100， m 不超過 4000。

輸出：第一行輸出可以到達的點數，第二行輸出與 s 的距離總和。

範例一輸入： <pre> 7 6 1 5 1 1 3 1 4 2 3 4 6 6 0 </pre>	範例一輸出： <pre> 4 7 </pre>
範例二輸入： <pre> 3 2 0 1 2 2 1 </pre>	範例二輸出： <pre> 0 0 </pre>

這一題就是要在有向圖中從一點 s 做 BFS 就可以計算可以到達的點，以及 s 到這些點的距離。因為點數不多，我們同時示範鄰接矩陣與鄰接串列的做法，以下先看矩陣。這個程式就直接照 BFS 的演算法寫，沒有太多需要進一步說明的，因為適用鄰接矩陣儲存的，所以在找鄰居時，必須跑一個迴圈檢查所有的點。

```

// P-7-1, using matrix
#include <bits/stdc++.h>
using namespace std;
#define N 102
int n, m, s;
int adj[N][N]={0};
int d[N]; // distance

void bfs(int s) {
    bool visit[N] = {false};
    queue<int> Q;
    Q.push(s);
    visit[s]=true, d[s]=0;
    int cnt=0, totald=0;
    while (!Q.empty()) {
        int v=Q.front();
        Q.pop();
        for (int i=0;i<n;i++) {

```

```

        if (adj[v][i] && !visit[i]) {
            d[i]=d[v]+1;
            visit[i]=true;
            Q.push(i);
            cnt++; // num of reachable
            totald+=d[i]; // total distance
        }
    }
}
printf("%d\n%d\n", cnt, totald);
return;
}

int main() {
    scanf("%d%d%d", &n, &m, &s);
    for (int i=0; i<m; i++) {
        int from, to;
        scanf("%d%d", &from, &to);
        adj[from][to]=1;
    }
    bfs(s);
    return 0;
}

```

再來看鄰接串列的寫法，惟一的差別只在圖的儲存方式，因為用 vector 來存，所以在找鄰居時可以很簡單的使用 range-based 的寫法

```
for (int u: adj[v]) { }
```

請注意這一題的輸入雖然兩點之間可能有多個邊而且可能有自己走到自己的邊（自迴圈），但對 BFS 完全沒有影響，所以程式中沒有去判別這些狀況。

```

// P-7-1, using adjacency list
#include <bits/stdc++.h>
using namespace std;
#define N 102
int n, m, s;
vector<int> adj[N];
int d[N]; // distance

void bfs(int s) {
    bool visit[N] = {false};
    queue<int> Q;
    Q.push(s);
    visit[s]=true, d[s]=0;
    int cnt=0, totald=0;
    while (!Q.empty()) {
        int v=Q.front();
        Q.pop();
        for (int u: adj[v]) {

```

```

        if (!visit[u]) {
            d[u] = d[v]+1;
            visit[u] = true;
            Q.push(u);
            cnt++; // num of reachable
            totald += d[u]; // total distance
        }
    }
}
printf("%d\n%d\n", cnt, totald);
return;
}

int main() {
    scanf("%d%d%d", &n, &m, &s);
    for (int i=0; i<m; i++) {
        int from, to;
        scanf("%d%d", &from, &to);
        adj[from].push_back(to);
    }
    bfs(s);
    return 0;
}

```

BFS 容易寫錯的地方是 `visit[]` 沒有設定好或是忘了檢查，那有可能會造成一個點被重複的放進 `queue` 中，而在執行時發生錯誤（例如無窮迴圈）。

7.2.3. DFS

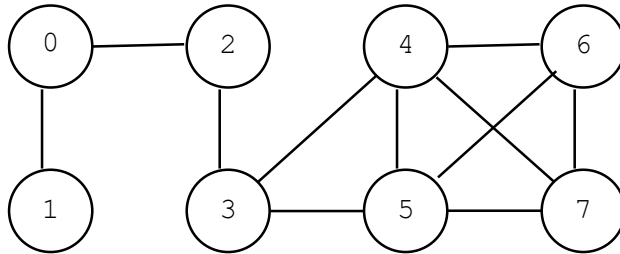
除了 BFS 之外，另外一個常用的圖的搜尋演算法是深度優先搜尋 (DFS, Depth-First Search)。BFS 像水面掀起的水波，從中心點往外一圈一圈的擴散，DFS 通常是以遞迴的概念來說明：「先從一個鄰居出發探索所有可以探索的點，如果還有剩下，則在從下一個鄰居出發，重複此步驟，直到所有鄰居嘗試完畢。」遞迴的演算法通常比較簡潔但不易了解，DFS 也不例外。

```

初始化對所有點 v，visit[v]=false;
DFS(s) {
    visit[s] = true;
    for each v ∈ adj[s] do
        if (visit[v] == false) then
            v.p = s; // 呼叫 v 的點就是 v 的 parent
            DFS(v);
        end if
    end for
    return;
}

```

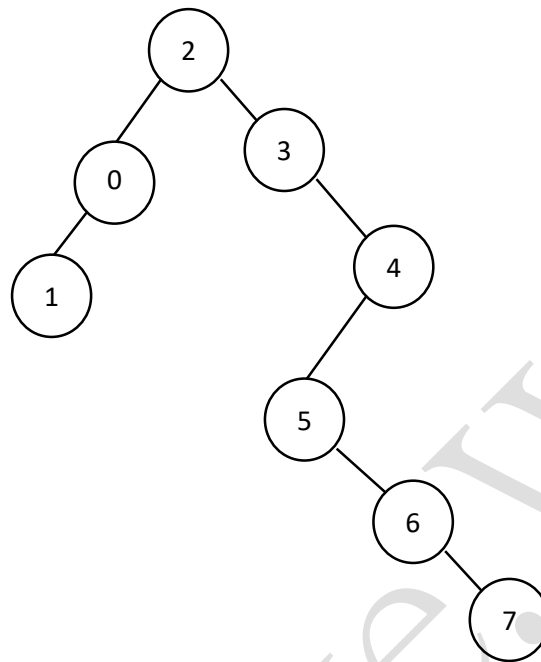
我們實際看一下 DFS 在下面這個無向圖上的運作狀況，會比較了解此演算法。假設一樣從 2 出發。



演算法運作過程如下 (假設鄰居的檢查順序依照編號)：

- A. DFS(2)：2 有兩個鄰居{0, 3}，而且都未曾被拜訪，先去 0，留著 3 未檢查；
- B. DFS(0)：0 有兩個鄰居{1, 2}，先去 1；
- C. DFS(1)：1 只有個鄰居{0}，但是 0 已經被拜訪過，所以 return 回到 B；
B 是從 0 探訪它的鄰居，本來先去 1，現在從 1 回來了，所以檢查下一個鄰居 2，但 2 是已經被拜訪過的，所以也結束，return 回到 A；
A 是從 2 探訪它的鄰居，本來先去 0，現在從 0 回來了，所以檢查下一個鄰居 3，3 尚未被拜訪，所以去 3；
- D. DFS(3)：3 有 3 個鄰居{2, 4, 5}，先檢查 2，被拜訪過；再檢查 4，沒拜訪過，所以去 4 (留著 5 還沒檢查)；
- E. DFS(4)：4 有 4 個鄰居{3, 5, 6, 7}，先檢查 3，被拜訪過；再檢查 5，沒拜訪過，所以去 5 (留著 6, 7 還沒檢查)；
- F. DFS(5)：5 有 4 個鄰居{3, 4, 6, 7}，先檢查 3 與 4，都被拜訪過；再檢查 6，沒拜訪過，所以去 6 (留著 7 還沒檢查)；
- G. DFS(6)：沒被拜訪過的鄰居只有 7；
- H. DFS(7)：沒有沒被拜訪過的鄰居，所以 return 回到 G (6 號點)；
G 沒有剩下沒拜訪過的鄰居，所以 return 回到 F (5 號點)；
F 本來留著 7 尚未檢查先去 6，所以 6 回來時檢查 7，但此時 7 已經被拜訪過了！！所以也沒有剩下沒拜訪過的鄰居，也 return 回到 E (4 號點)；
E 是 4 號點先去 5，留著 6 與 7 尚未檢查，所以從 5 號點回來會去檢查 6 與 7，但此時兩點都已經被拜訪過了，所以就 return 回到 D (3 號點)；
類似地，3 號回到 2 號，然後就結束了整個探訪的過程。

如果我們把每個點被誰呼叫當作它的 parent，然後把每點與它的 parent 連起來，一樣可以得到一棵樹，這棵樹是由 DFS 走出來的，所以就稱為 DFS tree (如下圖)，它可以讓我們了解 DFS 運作的過程。



回顧一下 DFS 的過程：從 2 出發，拜訪 0，拜訪 1，回到 0，回到 2，拜訪 3，拜訪 4，拜訪 5，拜訪 6，拜訪 7，回到 6，回到 5，回到 4，回到 3，回到 2，結束。所以它的行為是如果還能 (更深入) 往前走，就深入往前，直到沒有沒拜訪過的，就回頭。所以深度優先搜尋的名字是這麼來的。

- DFS 通常寫成遞迴形式，所以比較好寫，但有遞迴的缺點，在某些系統環境上，遞迴的深度會受到限制。
- DFS 與 BFS 一樣可以計算到達的點與無向圖的連通區塊，但是不能算起點到各點的距離。
- DFS 在進階的圖論演算法中有很多重要的應用，但大多超過了這份教材的難度，所以沒有在這教材中說明。

以下是一個簡單的例題來示範 DFS 的寫法。

P-7-2. 開車蒐集寶物

參加一個蒐集寶物的遊戲，你拿到一個地圖，地圖上有 n 個藏寶點，每個藏寶點有若干價值的寶物，由於你的團隊是最頂尖的，只要能到達藏寶點一定可以取得該藏寶點的寶藏。從地圖上可得到一共有 m 條道路，每條道路連接兩個藏寶點，而且每條道路

都是雙向可以通行的。在遊戲的一開始，你可以要求直升機將你的團隊運送到某個藏寶點，而且你可以獲得一部車與充足的油料，但是直升機的載送只有一次，所以你必須決定要從哪裡開始才可以獲得最多的寶藏總價值。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 與 m ，代表藏寶地點數與道路數，地點是以 $0 \sim n-1$ 編號，第二行 n 個非負整數，依序是每一個地點的寶藏價值，每個地點的寶藏價值不超過 100。接下來有 m 行，每一行兩個整數 a 與 b 代表一個道路連接的兩個地點編號。 n 不超過 $5e4$ ， m 不超過 $5e5$ 。兩點之間可能有多條道路，有些道路的兩端點可能是同一地點。

輸出：最大可以獲得的寶藏總價值。

範例一輸入： <pre> 7 6 5 2 4 2 1 1 8 5 1 1 3 1 4 2 0 2 0 3 3 </pre>	範例一輸出： <pre> 9 </pre>
範例二輸入： <pre> 3 2 2 1 5 1 0 0 1 </pre>	範例二輸出： <pre> 5 </pre>

範例一說明：降落在 0 可以獲得 0 與 2 的寶藏，價值為 $5+4=9$ 。

範例二說明：降落在 2 可獲得價值 5。

輸入的是一個無向圖，每個點有個權重，要找出最大權重的連通區塊，所以用 DFS 或 BFS 都可以，這一題我們用來示範 DFS，所以下面是 DFS 的寫法。這一題的點數很大，所以必須使用連接串列的方式來存圖，主程式主要在讀取圖的資料，因為是無向圖，所以讀到一個邊 (u, v) 時，必須兩點互相加為鄰居。DFS 函式的部分可以看得出遞迴的寫法真的要比 BFS 簡潔，這裡我們要找的是連通區塊的權重總和，所以，對於每一次 DFS，我們讓他回傳可以到達點的權重和就可以了，因為有判斷 `visit`，所以一個點不會被走訪兩次，權重也就不會被重複計算了。

```

// P-7-2, max weight component, adjacency list
#include <bits/stdc++.h>
using namespace std;
#define N 50100
int n, m, s;
vector<int> adj[N];
int w[N]; // weight of vertices
bool visit[N] = {false};

// return the total weight can be reached from s
int dfs(int s) {
    int weight=w[s];
    visit[s]=true;
    for (int u: adj[s]) {
        if (!visit[u])
            weight += dfs(u); // total weight
    }
    return weight;
}

int main() {
    scanf("%d%d", &n, &m);
    int total=0;
    for (int i=0;i<n;i++) {
        scanf("%d", &w[i]);
        total += w[i];
    }
    for (int i=0; i<m; i++) { // each edge
        int u,v;
        scanf("%d%d", &u, &v);
        adj[u].push_back(v); // v is a neighbor of u
        adj[v].push_back(u); // u also a neighbor of v
    }
    int maxw=0; // max total weight
    for (int v=0; v<n; v++) {
        // search the weight of component of v
        if (!visit[v])
            maxw=max(maxw,dfs(v));
    }
    printf("%d\n", maxw);
    // for checking
    fprintf(stderr, "ans=%d, total=%d\n",maxw, total);
    return 0;
}

```

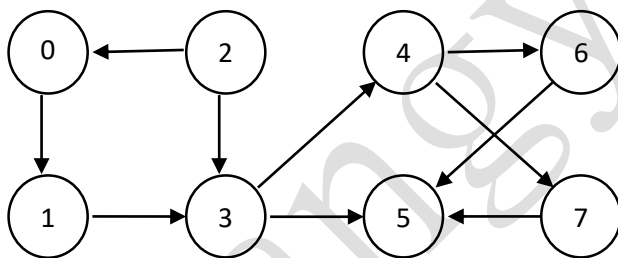
這一題當然也可以用 BFS 來作，有興趣的人可以自己練習試試，範例程式中有一支 BSF 的作法，因為只是標準的 BFS，這裡就不多說明了。

7.2.4. DAG 與 topological sort

DAG(directed acyclic graph)是指一種沒有環路的有向圖，DAG 是應用很多的一個圖的類別，我們在上一章動態規劃中簡單說明過 DAG 可以用來描述狀態轉移，其實不只是 DP 的狀態，在很多前置關係的問題上都有類似的情形，例如一件計畫分成很多工作，某些工作必須在某些工作完成之後才能進行。我們可以將每個工作看成一個點，若 u 是 v 的前置工作，就在圖上加入一個 (u, v) 有向邊，這樣建立出來的圖必須是個 DAG，否則就會產生雞生蛋蛋生雞的矛盾而有某些工作無法完成。

對於一個 DAG，我們最常須要找的就是一個拓樸順序(topological sort)，所謂拓樸順序是將所有的點排成一個序列，使得所有的有向邊都是從前往後，而沒有從後往前的邊。拓樸順序顯然不惟一，但通常只要找出其中一個就可以了。

以下圖為例來找一個拓樸順序，因為前置關係是有遞移性的，前置的前置也必須是前置，我們可以看得出來，2 一定要放在第一個，相同的 5 一定是最後一個。因為 3 要放在右邊所有點前，所以最前面的四個點一定是 $(2, 0, 1, 3)$ 。置於右邊的四個點，因為 4 必須在 6 與 7 之前，所以 3 後面只能接 4，但 6 與 7 不一定誰先誰後，所以我們知道 $(2, 0, 1, 3, 4, 6, 7, 5)$ 是一個拓樸順序，而 $(2, 0, 1, 3, 4, 7, 6, 5)$ 也是一個。



我們的重點要放在如何計算一個拓樸順序，常用的方法有兩個。其中一個是利用 DFS，假如在 DFS 時，對每一個點在完成探訪時(return 之前)，將該點的編號輸出，那麼將此輸出順序反轉就可以得到一個拓樸順序。證明不是很難，簡單的說，在執行 DFS(v) 時，所有 v 可以到達的點都是應該排在 v 之後的，而 DFS 會把這些點都拜訪完畢後才結束 v 的探訪。

DFS 的複雜度線性時間，所以用 DFS 來做拓樸順序應該很完美，但是它是遞迴的，很多時候我們不希望使用遞迴，例如 DP 的過程，因為不想(或是環境不允許)使用遞迴，所以才要找拓樸順序，所以我們需要一個非遞迴的演算法。這個演算法我們在 P-3-1 (樹的高度)時介紹過了，當時是用在找樹的一個 bottom-up sequence，事實上樹在指定方向(例如由下而上)後就是一個簡化的 DAG。以下我們看對任意 DAG 都可以找出拓樸順序的演算法。

這個方法很簡單，它的原理就是：只要是 $\text{in-degree}=0$ 的點，也就是沒有箭頭指向它的點，都是可以出列的點。以下是一個使用 adjacency list 的範例程式，第 6 行宣告一個整數陣列存每個點的 in-degree ，在輸入每一個邊的時候，於第 13 行計入各點的 in-degree 。第 15 行的陣列 `topo[]` 以及兩個變數 `head` 與 `tail` 是自製的佇列，因為我們想要保留拓撲順序，所以使用 STL 的 `queue` 反而不便。第 17~19 行的迴圈將 in-degree 為 0 的點放入佇列 (`topo[]`) 中，然後跑一個 `while` 迴圈直到佇列中沒有元素為止。在迴圈中，我們每次從佇列前端拿出一點 `v`，將 `v` 指向的每個點 `u` 的 in-degree 減一，若減一後降為 0，則將其放入佇列中。如果這是一個 DAG，`while` 迴圈跑完時，所有的點一定都處理過，否則便是有迴圈。程式的時間複雜度是 $O(n+m)$ ，因為每個點最多進入佇列一次，離開佇列時所花的時間是他的 out-degree ，而所有點的 out-degree 總和就是邊數。

```

00 //ch7 topological sort O(m+n), non-recursive
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 20
04 int main() {
05     vector<int> adj[N]; // adjacency list
06     int indeg[N]={0};
07     int n, m;
08     scanf("%d%d", &n, &m);
09     for (int i=0; i<m; i++) {
10         int from, to;
11         scanf("%d%d", &from, &to);
12         adj[from].push_back(to);
13         indeg[to]++;
14     }
15     int topo[N], head=0, tail=0; //queue
16     // push all indeg==0
17     for (int i=0; i<n; i++)
18         if (indeg[i]==0)
19             topo[tail++]=i; // push
20     while (head<tail) { // until queue is empty
21         int v=topo[head++]; // pop from queue
22         for (int u: adj[v]) {
23             if (--indeg[u] ==0)
24                 topo[tail++] = u;
25         }
26     }
27     // output
28     if (tail<n) printf("No a dag\n");
29     else {
30         for (int i=0; i<n; i++)
31             printf("%d ", topo[i]);
32         printf("\n");
33     }
34     return 0;
35 }

```

7.3. 例題與習題

這一節裡面我們提出幾個例題與習題，由於圖論演算法的題目雖多，但大部分超過 APCS 的範圍，所以這裡只有一些比較簡單的題目。有一些一維與二維陣列的題目其實用圖的概念會更容易了解，我們也放了一些這樣的題目。第一題是一個很簡單的二維陣列走訪的題目，沒有學過圖的也能作答，但用一些技巧會更簡單。

P-7-3. 機器人走棋盤 (APCS 201906)

有一個方格棋盤的每一個格子裡都標示了一個不同的整數，有一個機器人在此方格棋盤上行動，每一次只會移動到目前位置的上下左右四個相鄰格子其中之一。起點是數字最小的格子，每次移動則在可以移動位置中挑選數字最小的格子，但是走過的格子就不會再走，當無路可走的時候，機器人就會停下來。輸入方格棋盤中每個格子的數字，請模擬機器人走過的路徑，並輸出機器人走過的格子的數字總和。

以下是一個例子，輸入的 4×5 的方格內的數字如圖中所標示。在本例子中，機器人的起點會是 1，所走的路徑是 $1 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 14 \rightarrow 20 \rightarrow 21 \rightarrow 29 \rightarrow 30$ 。走到 30 的時候已經無路可走，所以機器人就停止了，而經過的數字總和是 132。

24	7	14	20	30
11	6	4	21	29
2	8	1	35	40
3	9	5	12	15

Time limit: 1 秒

輸入格式：輸入的第一行是兩個不超過 100 的正整數 m 與 n ，代表是一個 $m \times n$ 的方格棋盤，接下來有 m 行，每行 n 個數字，分別是方格棋盤由上而下，由左而右的數字。方格內的數字皆為不超過 $1e5$ 的非負整數，同一行數字之間以空白間隔。

輸出：機器人走過的格子中數字的總和。

範例一輸入：

```
2 7
6 8 7 2 1 4 5
9 3 10 11 12 13 14
```

範例一輸出：

```
36
```

範例二輸入： 4 5 24 7 14 20 30 11 6 4 21 29 2 8 1 35 40 3 9 5 12 15	範例二輸出： 132
範例一說明：機器人走的路線是 (1, 2, 7, 8, 3, 9, 6)。 範例二說明：此為題目敘述中的範例。	

這一題可以分成兩個部分：(1) 找出一個二維陣列中的最小值，以該位置為起點；(2) 模擬機器人的行走，對於某一個位置，找到上下左右四個鄰居中沒走過的最小值，而且要能辨識是否已經無路可走。第一個部分很簡單，處理二維陣列(矩陣)，我們可以搭配一個雙迴圈，一方面讀取矩陣內容，同時檢查最小值。為了要模擬機器人的行走位置，我們以一對變數(s_i, s_j)代表機器人所在位置在第 s_i 列 s_j 行，這裡我們採取矩陣行列的定位方式，使用平面座標的定位方式也可以。

為了要避免走到曾經走過的格子，我們可以採用圖形走訪的技巧，利用一個陣列 `visit[][]` 來記錄哪些格子被走過與否，另外為了預防出界，我們可以在原矩陣的外圍擺一圈虛擬的格子，並將他們都設定成已走過，這樣我們的程式碼會比較簡單。此外，每次我們要檢查四個鄰居，所以要寫四個 `if`，如果是可以走鄰近八方位的題目則要寫 8 個 `if`，這實在太囉嗦，所以我們把他寫成迴圈的樣子。請看以下範例程式，我們宣告了兩個長度為 4 的陣列 `di[]` 與 `dj[]`，裡面存放的其實是右下左上四個鄰居的列與行位移量。在檢查四個鄰居時，每一個鄰居的位置就是 $(s_i + di[k], s_j + dj[k])$ ，為了讓指令容易懂，不要太過複雜，所以另外設兩個變數 n_i 與 n_j 。

```
#include <bits/stdc++.h>
using namespace std;
#define N 102
#define oo 100001

int main() {
    int a[N][N];
    bool visit[N][N]={false};
    int di[4]={0,1,0,-1}, dj[4]={1,0,-1,0};
    int i,j,n,m;
    scanf("%d%d", &m, &n);
    // set boundary to visited
    for (i=0; i<=m+1; i++)
        visit[i][0]=visit[i][n+1]=true;
```

```

for (j=0; j<=n+1; j++)
    visit[0][j]=visit[m+1][j]=true;
// read input and find minimal
int si,sj,imax=oo; // (si,sj) is current position
for (i=1; i<=m; i++) for (j=1; j<=n; j++) {
    scanf("%d",&a[i][j]);
    if (a[i][j] < imax) {
        si=i, sj=j, imax=a[i][j];
    }
}
// start walking
int total=0;
while (1) {
    total += a[si][sj]; // goto (si, sj)
    visit[si][sj] = true; // mark visited
    imax = oo;
    for (int dir=0; dir<4; dir++) { // check 4 neighbors
        int ni=si+di[dir], nj=sj+dj[dir];
        if (!visit[ni][nj] && a[ni][nj] < imax)
            i=ni, j=nj, imax=a[ni][nj];
    }
    if (imax == oo) break; // no way to go
    si=i, sj=j;
}
printf("%d\n",total);
return 0;
}

```

很多棋盤上最少走幾步的問題，其實是圖的 BFS 的一種形式，例如象棋的車、馬、西洋棋的皇后，下面是一個這樣的問題。

P-7-4. 方格棋盤的最少轉彎數路線

有一個 $m \times n$ 的方格棋盤，每個格子可能是 0 或 1，其中 0 代表可以通過而 1 代表不能通過。現在有一個機器人從左上角編號 $(1, 1)$ 的格子出發，要到達右下角編號 (m, n) 的格子，每次可以往上下左右四個方向移動，我們要找到轉彎次數最少的路線。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 m 與 n ，代表格子棋盤的列數與行數，接下來有 m 行，每行是一個長度為 n 僅由 0 與 1 組成的字串，代表方格棋盤由上往下由左至右的內容，出發與目的格子必然是 0。 m 與 n 不超過 500。

輸出：最少的轉彎數。如果無法到達，則輸出 -1。

範例一輸入：

3 6

範例一輸出：

5

001010 101000 100010	
範例二輸入： 3 4 0110 0110 0010	範例二輸出： -1

把每一次轉彎當作走一步，這是最少步數的問題，所以可以用 BFS 來解。為了方便我們採取上一題類似的手法，在邊界放上不能走的標記，另外定義四個方向的位移。其他就是 BFS 的作法，停止條件包括可以走的已經走完或是目的位置已經達到。

```
// P-7-4 minimum turn
#include <bits/stdc++.h>
using namespace std;
#define N 510
#define oo 1000000

int main() {
    char a[N][N]; // chessboard
    int dis[N][N]; // min distance(turn)
    int di[4]={0,1,0,-1}, dj[4]={1,0,-1,0};
    int i,j,n,m;
    scanf("%d%d", &m, &n);
    // set boundary
    for (i=0; i<=m+1; i++)
        a[i][0]=a[i][n+1]='1';
    for (j=0; j<=n+1; j++)
        a[0][j]=a[m+1][j]='1';
    for (i=1; i<=m; i++)
        scanf("%s", &a[i][1]);
    for (i=1; i<=m; i++) for (j=1; j<=n; j++)
        dis[i][j]=-1; // not visited
    queue<pair<int,int>> Q; // visited (i,j)
    Q.push({1,1}); // starting position
    dis[1][1]=0;
    // start BFS
    while (!Q.empty() && dis[m][n]<0) {
        auto p=Q.front();
        Q.pop();
        int si=p.first, sj=p.second;
        for (int dir=0; dir<4; dir++) { // 4 directions
            int ni=si+di[dir], nj=sj+dj[dir];
            while (a[ni][nj]=='0') { // until touch 1
                if (dis[ni][nj]==-1) { // not visited
                    dis[ni][nj]=dis[si][sj]+1;
```

```

        Q.push({ni,nj});
    }
    ni+=di[dir];
    nj+=dj[dir];
}
}
if (dis[m][n]>0) dis[m][n]--; // count one more
printf("%d\n",dis[m][n]);
return 0;
}

```

下一個是 BFS 在一維數線上的問題，留為習題。

Q-7-5. 闖關路線 (APCS201910)

某個闖關遊戲上有一隻神奇寶貝與兩個可控制左右移動的按鍵。神奇寶貝被安置在僅可左右移動的滑軌上。滑軌分成 n 個位置，由左到右分別以 $0 \sim n - 1$ 表示。當遊戲開始時，神奇寶貝從位置 0 開始，遊戲的資訊包含 P 、 L 與 R 三個數字，其中 P 表示所須移至的目標位置， L 與 R 則分別表示每按一次左鍵或右鍵後，會往左或往右移動的格子數。此外，每一個位置 x 都對應一個瞬間移動位置 $S(x)$ ；每一次按鍵後，神奇寶貝會先依據按鍵往左或右移動到某個位置 x ，接著瞬間移動至 $S(x)$ 。某些點的瞬間移動位置等同原地點，也就是 $S(x) = x$ ，這些點稱為停留點。開始與目標位置都一定是停留點；此外，每個點的瞬間移動位置都一定是停留點（除非超出界外），也就是不會發生連續瞬間移動的情形。

遊戲的目標是以最少的按鍵數操作神奇寶貝由開始位置到達目標位置，此外，在移動過程中不可以超過滑軌的範圍 $[0, n - 1]$ ，否則算闖關失敗；某些點的瞬間移動位置也可能會超出滑軌的範圍，移動到這些點也會導致闖關失敗。

Time limit: 1 秒

輸入格式：輸入有兩行，第一行有 4 個數字，第 1 個為 n ，第 2 個為目標位置 P ，第 3 個為 L ，第 4 個為 R ，後三個數字皆為小於 n 之正整數，且 $2 \leq n \leq 1e6$ 。第二行有 n 個整數，依序是各點的瞬間移動位置 $S(0), S(1), \dots, S(n - 1)$ ，這些數字是絕對值不超過 $1e8$ 的整數。

輸出：輸出到達目標位置所需的最少按鍵數，如果無法到達目標位置，則輸出 -1 。

範例一輸入：

```

5 3 1 2
0 3 2 3 5

```

範例一輸出：

```

2

```

範例二輸入：	範例二輸出：
10 8 2 3 0 5 2 3 4 5 6 6 8 9	3

範例一說明：位置區間為 $[0, 4]$ ，目標位置為 3，左鍵往左移動 1 格，右鍵往右移動 2 格。到目標位置最少的按鍵數是 2 次：從起點 0 第一次按下右鍵會到達位置 2，因為 $S(2)=2$ ，因此停留在 2，第二次按下左鍵，會往左移一格到達 1，因為 $S(1)=3$ ，所以瞬間移動到 3，由於 3 是目標位置，所以就闖關成功了。

範例二說明：位置區間為 $[0, 9]$ ，目標位置為 8，按一次左鍵往左移動 2 格，按一次右鍵往右移動 3 格。到達目標位置的最少按鍵數是 3 次，其經過的路徑是：0，按右鍵 $\rightarrow$ 3（停留在 3），按左鍵 $\rightarrow$ 1（瞬間移動到 5），按右鍵 $\rightarrow$ 8（停留在 8，到達）。

在一條數線上，每次按左鍵可以目前位置移動到某處，按右鍵可以到達某處。要計算從起點到終點最少要按幾次鍵。起點固定在 0 的位置，而終點則是輸入時給定。移動的規則是：按下右鍵往右移 R 格，按下左鍵往左移 L 格，此外，每次按鍵後移動到位置 x 時，會立刻移動到 $S(x)$ 。

如果沒有瞬間移動，這一題幾乎是問 $xR - yL = P$ 的最小 $x+y$ 整數解，我們這裡不探討數學問題，在有瞬間移動的狀況下，要以計算來完成求解。他顯然是 BFS 的退化在一維數線上的狀況，基本上我們算出一步（一次按鍵）可以到達的點，然後算兩步的點，...，直到走到目標或是無路可走為止，甚至不需要了解 BFS 也可以推想出作法，但是有 BFS 的知識讓我們可以很容易的作出標準作法。

DAG 上的路徑問題

借用下一個例題，我們來學習一下 DAG 的路徑長度問題。圖的路徑長度問題是一群重要的問題，典型的題目是找最短路徑或最長路徑，根據圖的性質與條件，問題的難度不一，須要使用不同的演算法。目前為止我們會使用 BFS 計算無加權圖的最短路徑，如果邊上有權重（長度），那麼 BFS 無法求得正確的解，但是對於 DAG，我們可以用類似 BFS 的方法計算出最長與最短路徑。我們先以最短路徑為例，最長路徑的做法是相同的，這裡的最長路徑是指最長的簡單路徑。

假設要計算 DAG $G=(V, E, w)$ 上某點 s 到 t 的最短路徑。對任一點 v ，令 $d[v]$ 是 s 到 v 的最短路徑長度，回想一下在動態規劃時的狀態轉移，我們可以知道， $d[s]=0$ ；而對於 $v \neq s$ ，

$$d[v]=\min\{d[u]+w(u,v): \text{ for all } (u,v) \text{ in } E\}$$

其中這些 u 就是 v 的前置狀態。這其實只是一種很簡單的 DP，類似於 P-6-6 (方格棋盤路線) 中從左上走到右下的最大權重路線，只是只能往右往下的棋盤是一個很簡單的 DAG。有了這個遞迴式之後，計算的方向就是 DAG 的拓樸順序，所以只要找到拓樸順序後，計算 $d[]$ 值就可以了，另外，也可以一面找拓樸順序一面計算 $d[]$ 。

P-7-6. DAG 的最長與最短路徑 輸入一個 DAG 以及 s 與 t 兩點，計算 s 到 t 的最短與最長簡單路徑的長度。兩點之間可能有多個邊。 Time limit: 1 秒 輸入格式：第一行是兩個正整數 n 與 m，代表點數與邊數，點以 $0\sim n-1$ 編號，第二行兩個整數 s 與 t，接下來有 m 行，每行三個整數 u, v, w 代表一條有向邊 (u, v) 的長度是 w。n 不超過 $1e4$，m 不超過 $1e5$，w 的絕對值不超過 $1e4$。輸入保證是個 DAG。 輸出：第一行輸出最短路徑長度，第二行輸出最長路徑長度，如果不存在，兩者皆輸出 "No path"。	
範例一輸入： 5 6 0 4 0 2 3 0 3 1 2 1 -2 3 4 0 1 4 2 2 4 3	範例一輸出： 1 6
範例二輸入： 4 3 2 0 2 1 5 1 3 0 0 1 1	範例二輸出： No path No path

以下的範例程式中，我們採取一回合式的做法，也就是找拓樸順序的同時，就進行最短與最長路徑的計算。程式與前面計算拓樸順序的範例程式很像，不過這裡因為邊是有加權長度的，所以在儲存圖的鄰接串列時，我們需要儲存鄰居的編號以及到達該鄰居的邊長，程式中採用的方式是以 `pair<int,int>` 的結構來存一個邊，其中 `first` 是點的

編號而 second 是邊長。因為要計算最短與最長距離，我們用 $d[v]$ 與 $p[v]$ 代表起點 s 到達 v 點的最短與最長距離，只要跟著拓樸順序來作 DP 就可以了。是

```
//P-7-6 longest and shortest path on dag  $O(m+n)$ , non-recursive
#include <bits/stdc++.h>
using namespace std;
#define N 10010
#define oo 1000000000
int main() {
    vector<pair<int,int>> adj[N]; // (neighbor, length)
    int indeg[N]={0};
    int n, m, source, goal;
    scanf("%d%d", &n, &m);
    scanf("%d%d", &source, &goal);
    for (int i=0; i<m; i++) {
        int from, to, w;
        scanf("%d%d%d", &from, &to, &w);
        adj[from].push_back({to,w});
        indeg[to]++;
    }
    queue<int> Q;
    int d[N], p[N], cnt=0; // min and max distance
    for (int i=0; i<n; i++) d[i] = oo;
    for (int i=0; i<n; i++) p[i] = -oo;
    d[source] = p[source] = 0;
    // push all indeg==0
    for (int i=0; i<n; i++)
        if (indeg[i]==0)
            Q.push(i);
    while (!Q.empty()) { // until queue is empty
        int v=Q.front();
        Q.pop(); cnt++;
        for (auto e: adj[v]) {
            if (d[v] < oo) { // path exists
                d[e.first] = min(d[e.first], d[v] + e.second);
                p[e.first] = max(p[e.first], p[v] + e.second);
            }
            if (--indeg[e.first] ==0)
                Q.push(e.first);
        }
    }
    // output
    if (cnt!=n) fprintf(stderr, "Not a dag, %d < %d\n", cnt, n);
    if (d[goal]==oo) printf("No path\nNo path\n");
    else printf("%d\n%d\n", d[goal], p[goal]);
    return 0;
}
```

我們當然也可以用 top-down 的 DP，寫成遞迴的樣子，以下是範例程式。為了進行遞迴呼叫，我們儲存邊的方式是存 in-neighbor，也就是存每個點的前置點有哪些。其

他部分就只是簡單的 top-down DP 的寫法，我們將找 d[] 與 p[] 分成兩個函式，事實上當然可以寫在一個函式中。

```
//P-7-6 longest and shortest path on dag, recursive DP
#include <bits/stdc++.h>
using namespace std;
#define N 10010
#define oo 1000000000
vector<pair<int,int>> pred[N]; // (neighbor, length)
int d[N], p[N]; // min and max distance
// min distance from source

int mind(int t) {
    if (d[t] <= oo) return d[t]; // check memo
    d[t] = oo; // find minimum from each predecessor
    for (auto e: pred[t]) {
        int x=mind(e.first);
        if (x<oo) d[t] = min(d[t], x + e.second);
    }
    return d[t];
}

// max distance from source
int maxd(int t) {
    if (p[t] >= -oo) return p[t];
    p[t] = -oo;
    for (auto e: pred[t]) {
        int x=maxd(e.first);
        if (x>-oo) p[t] = max(p[t], x + e.second);
    }
    return p[t];
}

int main() {
    int n, m, source, goal;
    scanf("%d%d", &n, &m);
    scanf("%d%d", &source, &goal);
    // read predecessor of each node
    for (int i=0; i<m; i++) {
        int from, to, w;
        scanf("%d%d%d", &from, &to, &w);
        pred[to].push_back({from,w});
    }
    for (int i=0; i<n; i++)
        d[i] = oo+1; // >oo as un-computed
    for (int i=0; i<n; i++)
        p[i] = -oo-1; // <-oo as un-computed
    d[source] = p[source] = 0;
    d[goal] = mind(goal); // can be combined in one pass
    p[goal] = maxd(goal);
    if (d[goal]==oo) printf("No path\nNo path\n");
    else printf("%d\n%d\n", d[goal], p[goal]);
    return 0;
}
```

以下一個題目是 DAG 在計劃管理上的應用，AOV 是 Activity On Vertex 的縮寫，我們留做習題。

Q-7-7. AOV 最早完工時間 某個計劃有 n 項工作，以 $1 \sim n$ 編號，工作之間有前置關係，我們用一個點代表一項工作，以邊 (a, b) 表示 a 是 b 的前置工作，也就是說 a 完成之後，工作 b 才能開始。每一項工作 v 有一個需求的工作時間 $w[v]$，代表該工作至少需要耗時 $w[v]$ 才能完成。 本題要計算此計畫的最早完工時間，也就是計畫開始後，最早可以完成所有工作的時間。此外，對於某個工作，如果該工作的任何延誤都會讓整個計畫的最早完工時間延後，這個工作就稱為關鍵工作，否則稱為非關鍵工作。請計算那些工作是關鍵工作。 舉例來說，$n=3$，前置關係有 $(1, 3)$ 與 $(2, 3)$，代表工作 1 完成後才可以開始工作 3，相同的，工作 2 完成後才可以開始工作 3。需求工時為 $w[1]=1$，$w[2]=3$，$w[3]=4$。我們可以看出，工作 1 與 2 可以一起開始平行作業，在時間 1 時可以完成工作 1，時間 3 時可以完成工作 2，因此工作 3 最早可以開始的時間是 3，而最早的完工時間是 7。這三項工作中，工作 2 與 3 是關鍵工作，但工作 1 不是，因為工作 1 只要不花超過 3 的工時（延誤超過 2），並不會影響整個計畫的最早完工時間。 Time limit: 1 秒 輸入格式：第一行是兩個正整數 n 與 m，代表工作數與前置關係數，點以 $1 \sim n$ 編號，第二行是 n 個正整數，依序代表每一個工作的需求工時 $w[v]$，接下來有 m 行，每行兩個整數 u 與 v，代表 u 是 v 的前置工作。n 不超過 $1e4$，m 不超過 $1e5$，w 不超過 $1e3$。輸入保證有解。 輸出：第一行輸出最早完工時間，第二行輸出那些工作是關鍵工作，輸出時依照工作的編號由小到大，相鄰兩數字之間以一個空白分格。	
範例一輸入： 3 2 1 3 4 1 3 2 3	範例一輸出： 7 2 3
範例二輸入： 4 4 1 2 3 4	範例二輸出： 8 1 3 4

1 2	
1 3	
1 4	
3 4	

下一個也是習題。

Q-7-8. 小寶的著色問題

小寶有個布娃娃，娃娃會變化。娃娃身上有一些圖案，這些圖案是由一些圓圈和一些線條組成，每個線條連接著某兩個不同的圓圈。小寶很喜歡著色，他決定要把這些圓圈塗上紅色或藍色，他覺得有線條相連的圓圈最好塗上不一樣的顏色，但是因為圓圈很多，他不知道能不能夠完成這樣的想法，請你幫他計算看看是否有辦法完成這樣的著色。

Time limit: 1 秒

輸入格式：第一行有一個整數 T ，代表接下來有 T 張圖的資料要計算。每張圖資料的第一行是兩個整數 n 與 m ，代表有 n 個圓圈與 m 個線條，第二行有 $2m$ 個整數，每兩個一組代表一根線條所連接的兩個圓圈編號，圓圈是以 $0 \sim n-1$ 編號。 n 不超過 $1e4$ ， m 不超過 $1e5$ 。 T 不超過 20。

輸出：依序每一行輸出一張圖是否可以正確著色，如果是，則輸出 yes，否則輸出 no。

範例一輸入：	範例一輸出：
2	yes
2 0	no
4 3	
0 3 3 2 2 0	

範例說明：第一張圖 $n=2$ ， $m=0$ ，表示有兩個圓圈而沒有線條，可以正確著色。第二張圖有 4 個圓圈與 3 根線條，3 根線條是 $(0, 3)$ 、 $(3, 2)$ 與 $(2, 0)$ ，這 3 個點如果只用 2 種顏色，不管如何上色都會有同色的相連。

提示：把圓圈看成圖的點，線條看成邊。每個點與他的鄰居必須不同色，我們可以用 BFS 或 DFS 做圖的走訪與著色，過程中檢查是否有衝突發生。

7.4. 補充教材 (*)

在這一節中，我們挑選一些還算常用也不太困難的圖論演算法當作補充教材，包括：Dijkstra's 演算法、併查集 (Union and Find)，以及最小生成樹 (Minimum spanning tree)。

7.4.1. Dijkstra 演算法

我們前面介紹了無加權圖上面以 BFS 計算最短距離的方法，對於 DAG 我們也可以沿著拓撲順序來計算距離，但是應用問題中，更常見的是邊上有加權也可能有環路的圖。Dijkstra 演算法是用在一個加權圖上計算一點到所有點的最短路徑的演算法，圖可以有向的或無向的，但是邊上的權重必須是非負的。

假設輸入的圖是 $G=(V,E,w)$ 與一個起點 s ，我們要計算 s 到 V 中每一點的一條最短路徑。演算法運作過程中，我們以 $d[v]$ 紀錄目前已知 s 到 v 的最短距離，集合 Q 表示最短距離尚未確定的點，演算法如下：

```
// Dijkstra 演算法，輸入： $G=(V,E,w)$  與一個起點  $s$ 
初始化： $d[s]=0$ ；對其他  $v$  點， $d[v]=\infty$ ； $Q=V$ ；
while 尚有未確定距離的點 ( $Q$  is not empty) do
    在  $Q$  中找出  $d[]$  最小的點  $v$ ；
    將  $v$  移出  $Q$  中； //  $d[v]$  確定為  $s$  到  $v$  的最短距離
    for each  $u$  in  $\text{adj}[v]$  and in  $Q$  do: //與  $v$  相鄰尚未確定距離者
        if  $d[v]+w(v,u) < d[u]$  then // 從  $v$  過來路更近
             $d[u] = d[v]+w(v,u)$ ；
             $\text{parent}[u] = v$ ； // 已知到達  $v$  之最短路徑的前一點
        end if
    end for
end while
```

演算法其實不困難，每次在尚未確定距離的點中，找出 $d[]$ 最小的 v ，因為邊長都不是負的，所以 $d[v]$ 的值不可能更短了，我們就將它確定，然後，用 $d[v]+w(v,u)$ 去嘗試更新與 v 相鄰的 u 點的距離。如果圖中有些點是從 s 走不到的，最後找出來的距離就是初設的無窮大。此外，最後 $d[v]$ 是最短距離，那麼要如何找路徑呢？我們在每次更新距離時，存了 $\text{parent}[v]$ ，它記錄著從 s 到 v 的最短路徑的最後一步，所以當計算完成時，由後往前一步一步回推就可以得到路徑經過的點的順序。名字取為 parent 的原因是：如果將每個點與它的 parent 連起來，我們會得到一棵以 s 為根的樹狀圖，

這個樹稱為 shortest-path tree，在此樹上，s 到任一點的路徑都是在原圖上的最短路徑。

我們把重點擺在如何實作，以下一個例題來當作說明，這個例題是無向圖，對有向圖的做法類似。

P-7-9. 最短路徑 (*)

輸入一個無向圖 $G=(V,E,w)$ ，其中點以 $0\sim n-1$ 編號，而邊的權重是非負整數。計算 0 號點到其他點的最短路徑長度。兩點之間可能有多個邊。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 與 m ，代表點數與邊數，接下來有 m 行，每行三個整數 u, v, w 代表一條無向邊 (u,v) 的長度是 w 。 n 不超過 $1e4$ ， m 不超過 $1e5$ ， w 是不超過 $1e4$ 的非負整數。

輸出：第一行輸出 0 到各點之最短路徑長度中的最大值，也就是在可以到達的點中，最短距離最大的是多少，第二行輸出有多少點無法從抵達。

範例一輸入： 7 6 0 2 3 0 1 1 2 3 4 1 4 0 3 4 2 5 4 3	範例一輸出： 4 1
範例二輸入： 3 3 2 1 5 1 0 0 0 1 1	範例二輸出： 5 0

(範例測資中前兩筆的 n 不超過 500。)

要做出 $O(n^2)$ 複雜度的 Dijkstra 演算法一點都不困難，例如在 dense graph 上以鄰接矩陣儲存時，我們直接依照演算法的步驟來寫，就可以得到以下的範例程式了。

```

// O(n^2) Dijkstra algorithm, dense graph using matrix
#include<bits/stdc++.h>
using namespace std;
#define N 510
#define oo 1000000000

int main() {
    int n, m, w[N][N], parent[N];
    // input data
    scanf("%d%d", &n, &m);
    if (n>510) {fprintf(stderr,"too large n\n"); return 0;}
    for (int i=0;i<n;i++) for (int j=0;j<n;j++)
        w[i][j]=oo;
    for(int i=0;i<m;i++) {
        int u,v,t;
        scanf("%d%d%d", &u, &v, &t);
        w[u][v]=w[v][u]=min(w[u][v],t); // maybe multiple edges
    }
    // initial
    int d[N], source=0;
    bool done[N]={false}; // whether the distance is converged
    for (int i=0;i<n;i++) {
        d[i]=oo;
        parent[i]=-1;
    }
    d[source]=0;
    while (1) { // converge one vertex each iteration
        // find min d[] of not-done vertex
        int v=-1, mind=oo;
        for (int i=0;i<n;i++) {
            if (!done[i] && d[i]<mind) {
                v=i;
                mind=d[i];
            }
        }
        if (v<0) break; // no more reachable vertex
        done[v]=true;
        // update neighbor
        for (int i=0;i<n;i++) {
            if (d[v]+w[v][i]<d[i]) { // shorter path to i
                d[i] = d[v]+w[v][i];
                parent[i]=v;
            }
        }
    }
    int maxd=-1, cnt=0, far;
    for (int i=0;i<n;i++)
        if (d[i]<oo) {
            if (d[i]>maxd) maxd=d[i], far=i;
        } else cnt++;
    printf("%d\n%d\n",maxd,cnt);
    /*
    while (far>=0) {
        fprintf(stderr, "%d ",far);
    */
}

```



```

        far=parent[far];
    }*/
    return 0;
}

```

要把時間複雜度降下來，除了儲存圖要改用鄰接串列之外，最重要的是每一回合如何很快地找到 $d[]$ 最小的點，這就需要借助資料結構的幫忙。因為每次找到一點之後，我們會更新其他點的 $d[]$ ，因此 $d[]$ 是會動態變化的，也就是說無法靠著一開始用排序來處理。因為每次都要找最小值，最直接的想法是啟用一個優先佇列 (PQ, priority queue)，但是這裡的困難還不只在於此，最大的問題在於每一回合可能有某些 $d[]$ 值會更動 (降低)。基本上 PQ 是以 heap 寫的，如果是自己寫的 heap，我們可以修改這個資料結構，讓它可以處理 decreasing key 的狀況，但是一般為了節省時間，我們會採用 STL 中提供的 PQ，可惜 STL 的 PQ 並不提供此功能，多數競賽選手採取的變通方法是以下“偷懶”的策略：

我們將 $(d[u], u)$ 的結構放在 PQ 中，當我們要將 $d[u]$ 修改為 x 的時候，我們並不從 PQ 中刪除或更新該筆資料，而是直接將 (x, u) 放入 PQ 中。這樣一來，對於某些點，PQ 中可能會有多筆資料，但是無論如何，最小的會先出來，因為我們只做 decreasing key，所以最先出來的是最新的資料。需要額外處理的事情是，當我們每一回合找最小值時，需要檢查出來的最小值點是否已經處理完畢了，如果是，則直接丟掉它。

另外一個比較小的問題是 PQ 的比較函數自己寫比較麻煩 (不好記)，所以最好是方法是運用 STL 中的 pair。將我們要比較的 key ($d[u]$) 放在第一欄位，點的編號放在第二欄位，這樣可以完全不必自己動手，請看以下的範例程式。第 25 行我們宣告一個 `pair<int,int>` 的 PQ，預計將 $(-d[u], u)$ 放入資料，使用負值的原因是讓最小值先出來 (default 是最大值出來)。重點在第 26 行的 while 迴圈，每一回合會確定一點的最短距離，但是在第 30 行我們會檢查出來的點是否已經出來過了，如果是，則直接不予理會，否則，出來的才是我們要的。第 33 行的 for 迴圈，我們檢查該點的鄰居，如果有更好的 $d[]$ 值就予以更新，如同前面說的，我們將更新後的直接壓入 PQ 中。時間複雜度是多少呢？分析的方式類似 BFS，因為每個點的鄰居數總和是 $O(m)$ ，而在 PQ 上的操作都在 $O(\log(n))$ 時間內，所以總時間是 $O(m \log(n))$ 。

如果不使用 PQ，另外一種可能的處理方法是使用 set，因為 set 也可以找最小值，每次的變動都可以藉著刪除與新增來完成，我們在範例程式中有另外一支是以 set 來做的，留給有興趣的人自行參考，這裡就不多做明說了。因為要使用到動態資料結構，Dijkstra 演算法要在 APCS 中出現的機會應該是不大，或者給你演算法讓你模擬一個 $O(n^2)$ 的方法。

```

00 // Dijkstra algorithm O(mlogn) using PQ
01 #include<bits/stdc++.h>
02 using namespace std;
03 #define N 10010
04 #define oo 1000000001
05
06 vector<pair<int,int> > adj[N]; //Adjacency list
07 int d[N], parent[N];
08 bool done[N]={false};
09
10 int main() {
11     int n, m;
12     // input data
13     scanf("%d%d", &n, &m);
14     for(int i=0;i<m;i++) {
15         int u,v,t;
16         scanf("%d%d%d", &u, &v, &t);
17         adj[u].push_back({v,t});
18         adj[v].push_back({u,t});
19     }
20     // initial
21     int source=0;
22     for (int i=0;i<n;i++) d[i]=oo;
23     for (int i=0;i<n;i++) parent[i]=-1;
24     priority_queue<pair<int,int>> PQ; // (-d[], vertex)
25     PQ.push({d[source]=0,source});
26     while (!PQ.empty()) { // converge one vertex each iteration
27         auto p=PQ.top(); // find min d[] of not-done vertex
28         PQ.pop();
29         int v=p.second;
30         if (done[v]) continue; // already done, discard
31         done[v] = true;
32         // update neighbor
33         for (auto e: adj[v]) {
34             int u=e.first, w=e.second;
35             //if (done[u]) continue; // not necessary
36             if (d[v]+w < d[u]) { // shorter via v
37                 d[u] = d[v]+w;
38                 parent[u] = v;
39                 PQ.push({-d[u], u}); // push a better d[] of u
40             }
41         }
42     }
43     int maxd=-1, cnt=0, far;
44     for (int i=0; i<n; i++)
45         if (d[i]<oo) {
46             if (d[i]>maxd) maxd=d[i], far=i;
47             } else cnt++;
48     printf("%d\n%d\n", maxd, cnt);
49     /*
50     while (far>=0) {
51         fprintf(stderr, "%d ",far);
52         far = parent[far];
53     }*/
54     return 0;

```

7.4.2. 併查集(Union and Find)

併查集是一種資料結構，用來處理一群不相交的集合 (disjoint set)，說是一種資料結構，應該是一套資料儲存方式與操作的演算法。併查集處理的資料是不相交的集合，如同它的名字，提供的主要操作有兩個：合併兩集合以及查詢某個元素在哪個集合中。

併查集是一種樹狀圖的觀念，但是實作時完全以陣列來做就可以，也就是心中有樹但程式中無樹，寫起來非常容易，不容易寫錯，效率又高。為了方便說明，我們以人與社團來比喻元素與集合。假設元素的編號是 $0 \sim n-1$ ，我們只要一個陣列就可以，依循樹狀圖的概念，陣列就叫做 `parent[]`。每一個人都會屬於某個社團，但不會屬於兩個社團。除了社長，每個人 i 都有一個直接領導，就是他的衣食父母 `parent[i]`，社長就是社團的代表人，為了方便，若 i 是社長，我們令 `parent[i] = -x`，其中而 x 是該社團中的總人數。我們可以用下列方式來操作合併與查詢：

```
// 最簡單的 union and Find
find(x): // 給一個人的編號，回傳 x 所在社團的社長編號
    while (parent[x] >= 0) do
        x = parent[x]; // 一路往上找，直到社長
    end while
    return x;

union(x,y): // 合併 x 與 y 所在的集合
    r1 = find(x); // 找到 x 的社長老大
    r2 = find(y); // 找到 y 的社長老大
    if (r1 != r2) then // 不同社長才要合併
        parent[r2] = r1; // 一個社長拜另外一個社長當社長老大
    end if
```

這個演算法很簡單，查找時就一路沿 `parent[]` 往上走，直到走到社長 (代表元素) 為止。合併的時候，先檢查兩者是否是同一社團，如果是就不需要做任何事，如果不是，就把其中一個社團社長的衣食父母改為另外一個社長，這就為成了合併的動作。這個方法雖然簡單，但是有個缺點，在一連串合併之後，可能有些人離社長關係非常遠，想像每次把一個社團都併入一個一人社團，這就如同社團每次都來一個空降的新人當新的社

長，每併一次，最底下的那個人離社長的層級就增加一，這樣在查找的時候可能會很浪費時間。

改進的方法是這樣：每次合併時，把人數少的社團併入人數多的社團，也就是人多的社團社長當合併後的新社長。事實上，如果依照這個規定，社團的層級就不會超過 $\log(n)$ 。證明：如果合併造成你離社長的層級增加 1 的話，表示你併入一個比原來更大的社團，也就是你的社團人數至少增加一倍，如果你離社團的層級為 k 表示社團至少有 2^k 個人，在總人數只有 n 的前提下，層級不會超過 $\log(n)$ 。

其實做到這個方法，我們的併查集效率就會很不錯了，但還有另外一招來提高效率，這招數稱為路徑壓縮，做法是這樣：每當我們要做 `find()` 的時候，除了找到社長之外，我們“順便”把尋找過程中碰到的那些人的 `parent` 直接改設成社團社長。這樣做似乎看不出直接的效果，事實上它是個前人種樹後人乘涼的概念，這次多花一點時間，但後續再做查找時，因為路徑都縮短了，就會比較快。這個效率的分析很不容易，基本上有人分析出，如果有一連串的合併與查詢，在以上這樣的方法之下，每一個操作所需要的均攤時間幾乎是 $O(1)$ 。詳細的分析超過我們的範圍，這裡不多解釋了，以下是修改後的演算法，請留意社長的 `parent[]` 是存社團人數的負值，所以大小關係是顛倒的。

```
// 優化後的 union and Find
find(x): // 回傳 x 所在集合的代表元素並且做路徑壓縮 (遞迴版)
    if (parent[x]<0) then
        return x;
    end if
    parent[x] = find(parent[x]); // 遞迴, top-down DP
    return parent[x];

union(x,y): // 合併 x 與 y 所在的集合
    r1 = find(x);
    r2 = find(y);
    sum = parent[r1] + parent[r2]; // -總人數
    if (parent[r1] < parent[r2]) then
        parent[r2] = r1;
        parent[r1] = sum;
    else if (parent[r1] > parent[r2]) then
        parent[r1] = r2;
        parent[r2] = sum;
    end if
```

前面的例題 P-7-2 中，我們要找權重總和最大的連通區塊，當時是用 BFS 或 DFS 來做，事實上也可以用併查集來做，每讀到一個邊就把兩端點所在的連通區塊聯集起來，以下是這樣寫的範例程式。

```
// P-7-2 max component by find and union;
#include <bits/stdc++.h>
using namespace std;
#define N 50010
int p[N];
// find root
int rfind(int u) {
    if (p[u] < 0) return u;
    return p[u] = rfind(p[u]);
}

int main() {
    int m, n, w[N];
    scanf("%d %d", &n, &m); // printf("%d: ", n);
    for (int i=0; i<n; i++)
        scanf("%d", w+i);
    for (int i=0; i<n; i++)
        p[i] = -1; // parent<0 is a root
    for (int i=0; i<m; i++) { // read an edge and union
        int r1, r2, u, v;
        scanf("%d%d", &u, &v);
        r1 = rfind(u); // root of u
        r2 = rfind(v); // root of v
        if (r1==r2) continue; // same set
        if (p[r1] < p[r2]) {
            p[r1] += p[r2];
            p[r2] = r1;
            w[r1] += w[r2];
        } else {
            p[r2] += p[r1];
            p[r1] = r2;
            w[r2] += w[r1];
        }
    }
    int ans=0;
    for (int i=0; i<n; i++)
        if (p[i]<0 && w[i]>ans)
            ans = w[i];
    printf("%d\n", ans);
    return 0;
}
```

如同前面走訪的問題，連通區塊的概念也用在二維陣列。

P-7-10. 挖坑跳 (@@) (TOI 入營考)

小雨喜歡在土堆上挖坑，在坑裡注入水之後，然後玩跳水坑的遊戲。一個庭院被畫分成 $m \times n$ 個相同大小的矩陣方格，每一個格子可能是土堆或水坑，一個水坑格子與其上下左右四個方向的水坑格子被視為連通的同一個水坑。我們需要計算有多少個水坑以及最大的水坑佔多少個格子。除了一開始的土堆與水坑，小雨每次會指定將某個格子變成水坑，如果這個方格是新挖出來的水坑，有可能把附近的水坑連在一起。小雨一共挖了 k 次，請你計算出每一次挖了之後的水坑數與最大水坑面積。

土堆與水坑的資訊可以看成一個二維矩陣，以 0 表示土堆，而 1 表示水坑，位置的標示方式以左上角為 $(1, 1)$ ，右下角是 (m, n) 。以下是一個 $m=3, n=5$ 的例子。一開始的水坑資料如下：

1	0	0	1	1
0	1	1	0	1
1	1	0	0	1

我們可以看出來有 3 個水坑：左上角只佔 1 格的水坑，左下有一個 4 格的水坑，以及右方有一個 4 格的水坑。假設現在小雨把 $(2, 4)$ 的土堆改成水坑（紅色位置），左下與右方的水坑便會連接成一個面積為 9 的水坑。

Time limit: 1 秒

輸入格式：第一行是三個整數，依序是 m, n 與 k ，接下來 m 行是土堆與水坑資料，每一行有 n 個 0 或 1 的數字，順序為由上而下，從左至右。最後一行有 $2k$ 個數字，依序每兩個代表一個被挖成水坑的位置 (i, j) ，如果該位置本來就是水坑，就代表沒有動作。 m 與 n 不會超過 500， k 不超過 20000。

輸出：輸出兩行，第一行是每次最大水坑面積的總和，第二行是每次水坑數量的總和。計算總和時包含一開始的狀態，所以最多有 $k+1$ 次，但如果該次所挖位置本來就是水坑，代表沒有動作，該次的結果不列入總和計算。

範例一：輸入

```
3 5 1
1 0 0 1 1
0 1 1 0 1
1 1 0 0 1
2 4
```

範例一：正確輸出

```
13
5
```

範例一說明：這是題目敘述中的例子，一開始有 3 個水坑，大小是 (1, 4, 4)，操作一次後變 2 個水坑，大小是 (1, 9)。最大水坑面積總和是 4+9=13，水坑數樣總和是 3+2=5。	
範例二：輸入 2 6 2 0 1 1 0 1 1 0 1 0 0 0 1 2 4 1 4	範例二：正確輸出 14 6
範例二說明：一開始有 2 個水坑，面積是 (3, 3)，第一次操作後有 3 個水坑，面積是 (3, 3, 1)，第二次操作後變成一個面積 8 的水坑。	

給一個二維的 0/1 矩陣，本題要找出連通的 1 區塊，連通的定義是上下左右四方位。初始找出各個連通區塊可以使用 BFS 或 DFS，但是題目還有後半部：有 k 次操作，每次會將某個位置設為 1，如果該位置原來就 1，當然沒事發生，但如果是 0 變成 1，區塊就會變動。這時候如果做 DFS 或 BFS 可能會太花時間，因為可能有 2 萬次操作，每次的區塊可能達到 25 萬點。因為每個點只會屬於某個區塊，使用併查集來維護區塊是好的選擇。

題目本來是二維矩陣，每一個格子需要一對 (i, j) 來表示，在以下的範例程式中，我們把所有的點 (i, j) 以 row-major 的方式轉成一維陣列的位置 $i * (n + 2) + j$ 。行數 n 加 2 的原因是我們把外圍多留了一圈當四個邊界，邊界在初值都設成 0 (土堆)。在一維轉換後，每個點的四個鄰居的位移就是 (+1, -1, +n, -n)，其中 n 已經過加 2 的修正。範例程式中一共有三個副程式，rfind() 與 combine() 是併查集使用的，另外 dfs() 用在出始找連通區塊，它除了將走訪到的區塊標記為代表點的編號之外，也回傳區塊的大小。程式碼雖然有點長，但是如果了解 DFS 與併查集，應該不難了解。

```
// P-7-10 components in matrix, transform to 1d
// a cell (i,j) is indexed by i*(n+2)+j
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
#define N 510
char a[N*N]={0}; // a 0/1 matrix
int p[N*N], di[4]={1,-1}; // parent and index difference
int n_area=0, max_area=0;
// find root
int rfind(int u) {
```

```

    if (p[u] < 0) return u;
    return p[u] = rfind(p[u]);
}
// find component, return size
int dfs(int v, int root) {
    p[v]=root;
    int cnt=1;
    for (int dir=0;dir<4;dir++) {
        int u=v+di[dir];
        if (a[u] && p[u]==-1)
            cnt += dfs(u, root);
    }
    return cnt;
}
// combine two area, p[] = -size
void combine(int u,int v) {
    int r1 = rfind(u); // root of u
    int r2 = rfind(v); // root of v
    if (r1==r2) return; // same set
    max_area=max(max_area, -p[r1]-p[r2]);
    n_area--;
    if (p[r1] < p[r2]) {
        p[r1] += p[r2];
        p[r2] = r1;
    } else {
        p[r2] += p[r1];
        p[r1] = r2;
    }
    return;
}
int main() {
    int m, n, k;
    scanf("%d%d%d", &m, &n, &k);
    for (int i=1; i<=m; i++) for (int j=1;j<=n;j++) {
        p[i*(n+2)+j] = -1; // parent<0 is a root
    }
    for (int i=1; i<=m; i++) for (int j=1;j<=n;j++) {
        scanf("%d", &a[i*(n+2)+j]);
    }
    n+=2;
    di[2]=n, di[3]=-n; // next row and previous row
    int mn=(m+1)*n;
    // dfs for component
    for (int v=n; v<mn; v++) {
        if (a[v] && p[v]==-1) { // -1 is unvisited
            p[v] = - dfs(v, v);
            n_area++;
            max_area=max(max_area, -p[v]);
        }
    }
    LL total_n=n_area, total_max=max_area;
    //printf("%d %d\n", total_max, total_n);
    while (k--) { // read an cell
        int i, j, v;
        scanf("%d%d", &i, &j);

```



```

assert(i<=m && j<=n);
v=i*n+j;
if (a[v]==1) continue; // already 1, do nothing
a[v]=1; // set to 1
n_area++;
max_area=max(max_area, 1);
// try to combine 4 neighbors
for (i=0; i<4; i++) {
    if (a[v+di[i]]==0) continue;
    combine(v, v+di[i]);
}
total_n += n_area;
total_max += max_area;
}
printf("%lld\n%lld\n", total_max, total_n);
return 0;
}

```

下一個一維陣列的類似題，我們留做習題，因為測資設定的範圍小，這一題不使用併查集也可以通過。

Q-7-11. 紅白彩帶 (APCS201902)

一條彩帶被分成 n 個相同大小的格子，有的格子是紅色，有些則是白色。小軒拿到彩帶後開始塗顏色，每次會將一個白色格子塗滿紅色，然後小軒會算一算目前最長與最短紅色區域的長度佔了幾格，相鄰的紅色格子就屬於同一個紅色區域。小軒一共塗了 k 次，請你計算出每一次紅色區域的最大與最小長度，並輸出個別的總和。

彩帶可以看成一維陣列，以 0 表示白色，而 1 表示紅色。彩帶的格子從 1 開始由左而右依序編號，小軒每次將某個格子塗上紅色。以下是一個例子。

格子編號	1	2	3	4	5	6	7	8	9
剛開始的彩帶	0	1	1	0	0	1	0	1	0
一開始紅色區域最大的長度是2，最小的長度是1。									
第1次塗在第5格後	0	1	1	0	1	1	0	1	0
第1次塗完後最長的紅色區域有2塊，長度都是2，最小的長度是1。									
第2次塗在第1格後	1	1	1	0	1	1	0	1	0
第2次塗完後紅色區域的最大長度是3，最小長度是1。									
第3次塗在第7格後	1	1	1	0	1	1	1	1	0
第3次塗完後紅色區域的最大長度是4，最小長度是3。									

以這個例子而言，每一次(包含剛開始)紅色區域的最大長度總和是 $2+2+3+4=11$ ；而紅色區域的最小長度總和是 $1+1+1+3=6$ 。

Time limit: 1 秒

輸入格式：輸入有三行，第一行是兩個整數，依序是 n 與 k ，代表彩帶長度以及小軒塗色的次數， $n \leq 1e5$ 、 $k \leq 2e4$ 。第二行有 n 個 0 或 1 的數字，依序代表彩帶第 1 格至第 n 格的顏色，0 代表白色，1 代表紅色。第三行有 k 個數字，依序代表每一次塗紅色的格子編號，若 $k = 0$ ，則第三行為空行。同一行數字之間都是以空白間隔。小軒每次一定都塗在白色格子，而且紅色區域的長度不會超過 10100。

輸出：輸出兩行，第一行是每次紅色區域最大長度的總和，第二行是每次紅色區域最小長度的總和。

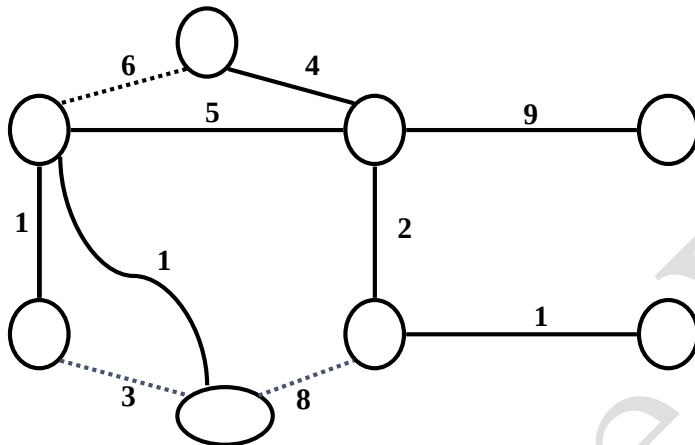
範例一：輸入 5 1 1 0 1 0 1 2	範例一：正確輸出 4 2
範例一說明：一開始最大長度是 1 而最小長度也是 1，塗在第二格之後最大長度是 3 而最小長度還是 1。最大長度總和為 $1+3=4$；最小長度總和為 $1+1=2$。	
範例二：輸入 9 3 0 1 1 0 0 1 0 1 0 5 1 7	範例二：正確輸出 11 6
範例二說明：此為題目中所舉的例子。	

7.4.3. 最小生成樹

樹是沒有環路的連通圖，對於一個圖 G ，它的一個生成樹 (spanning tree) 是指挑選 G 的一些邊，將所有點連成一個連通區塊，而挑選的邊形成的是一個樹。因為 n 個點至少要 $n-1$ 個邊才能形成一個連通區塊，而超過 $n-1$ 個邊必然會造成有環路，所以

spanning tree 一定恰好是 $n-1$ 個邊。 G 的最小生成樹 (minimum spanning tree) 是指邊的權重總和最小的生成樹。

假設我們有 n 個城市，要建設一個交通網路將 n 個城市連起來，如果圖 G 的每一個邊代表一條可以建設的道路，而邊上有一個非負的權重代表該道路的建設成本，那麼最小生成樹就是最低的建設總成本。下圖是一個例子，其中實線代表最小生成樹的邊，虛線代表沒有挑選的邊，圖中有 8 個點，一定剛好有 7 個邊。



如果是無加權圖，也就是圖的每個邊的權重皆相同，那麼最小生成樹的成本一定剛好是 $n-1$ ，BFS 與 DFS 都可以建構出一個最小生成樹。在一般狀況下，我們要處理的是加權圖。計算最小生成樹有多個演算法，目前比較常用的有兩個：Prim 演算法與 Kruskal 演算法。

先介紹 Prim 的演算法，這個演算法與 Dijkstra 演算法幾乎一模一樣，我們從任選一個點 s 開始，逐步建構我們要的樹 T ，每一回合會從尚未連進 T 的點中挑選一個點 v 以及一個已經在 T 中的點 u ，將 v 與邊 (u, v) 加入 T 中，而挑選的點與邊是滿足上面做法中成本最小的邊，也就是一點在 T 中一點不在 T 中的所有邊中，成本最小的邊。重複這個步驟直到全部的點都加入 T 中為止。以下演算法中我們計算成本，如果要建構出樹，就把每個點 v 與 $\text{parent}[v]$ 這個邊輸出即可。

```
// Prim 演算法，輸入： $G=(V, E, w)$  與一個任選一個起點  $s$ 

初始化： $T$  為空， $d[s]=0$ ；對其他  $v$  點， $d[v]=\infty$ ； $Q=V$ ； $\text{cost}=0$ ；
//  $Q$  是尚未加入  $T$  的點， $d[v]$  表示將  $v$  加入  $T$  的最小成本
while ( $Q$  is not empty) do
    在  $Q$  中找出  $d[]$  最小的點  $v$ ；
     $\text{cost} += d[v]$ ；//
    將  $v$  移出  $Q$  中；//  $v$  加入  $T$  中
```

```

for each u in adj[v] and in Q do: //與 v 相鄰尚未進入 T 者
    if w(v,u) < d[u] then // 從 v 連過來的成本更小
        d[u] = w(v,u);
        parent[u] = v; // u 加入 T 的最小成本是 w(v,u)
    end if
end for
end while

```

Prim 演算法與最短路徑的 Dijkstra 演算法只有比大小的 key 值不同，Dijkstra 是起點走過來的總距離，而 Prim 則只看最後與 parent 相連的邊長。實作時需要與 Dijkstra 一樣使用 PQ 的技巧，因為太相像了，我們就不多做說明，在後面的例題中，我們會給一個範例程式。

另外一個常用的是 Kruskal 演算法。這個做法也很簡單直覺，需要搭配的資料結構是併查集。演算法兩三句話就可以講完：將邊從小到大逐一考慮，如果一個邊加入不會形成環路就把它挑進來；否則將它捨棄。重複挑選邊直到沒有邊或者已經挑到 $n-1$ 個邊為止。

如果輸入的是連通圖，最小生成樹一定存在，否則全部的邊都挑完了仍無法形成生成樹。以下是演算法。

```

// Kruskal 演算法，輸入：G=(V, E, w)

初始化：設 T 為空；初始化一個併查集，每個點自成一個連通區塊；
將所有邊依照權重由小到大排序；
for each edge (u,v) do // 依權重由小到大的順序
    if u 與 v 不屬於同一個連通區塊 then // find()
        將 (u,v) 加入 T 中；
        將 u 與 v 所屬的連通區塊聯集； // union()
        // 如果 T 中已經有 n-1 個邊，可以提早結束
    end if
end for

```

下面一個簡單的例題用來介紹範例程式。

P-7-12. 最小生成樹 (*)

輸入一個無向圖 $G=(V,E,w)$ ，其中點以 $0\sim n-1$ 編號，而邊的權重是非負整數。計算 G 的最小生成樹的成本。兩點之間可能有多個邊。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 與 m ，代表點數與邊數，接下來有 m 行，每行三個整數 u, v, w 代表一條無向邊 (u,v) 的長度是 w 。 n 不超過 $1e4$ ， m 不超過 $1e5$ ， w 是不超過 $1e4$ 的非負整數。

輸出：輸出最小生成樹的成本，如果不存在則輸出 -1 。

範例一輸入： 8 10 0 1 6 0 2 4 1 2 5 2 3 9 1 4 1 1 5 1 2 6 2 4 5 3 5 6 8 7 6 1	範例一輸出： 23
範例二輸入： 4 3 2 1 5 1 0 0 0 2 1	範例二輸出： -1

範例一是本小節說明所用的例子。

(範例測資中前兩筆的 n 不超過 500。)

Prim 演算法幾乎與 Dijkstra 一模一樣，就不多做說明。

```
// Prim algorithm O(mlogn) using PQ
#include<bits/stdc++.h>
using namespace std;
```

```

#define N 10010
#define oo 1000000001

vector<pair<int,int> > adj[N]; //Adjacency list
int d[N], parent[N];
bool done[N]={false};

int main() {
    int n, m;
    // input data
    scanf("%d%d", &n, &m);
    for(int i=0;i<m;i++) {
        int u,v,t;
        scanf("%d%d%d", &u, &v, &t);
        adj[u].push_back({v,t});
        adj[v].push_back({u,t});
    }
    // initial
    int source=0;
    for (int i=0;i<n;i++) d[i]=oo;
    for (int i=0;i<n;i++) parent[i]=-1;
    priority_queue<pair<int,int>> PQ; // (-d[], vertex)
    PQ.push({d[source]=0,source});
    while (!PQ.empty()) { // converge one vertex each iteration
        auto p=PQ.top(); // find min d[] of not-done vertex
        PQ.pop();
        int v=p.second;
        if (done[v]) continue; // already done, discard
        done[v] = true;
        // update neighbor
        for (auto e: adj[v]) {
            int u=e.first, w=e.second;
            if (done[u]) continue;
            if (w < d[u]) { // shorter via v
                d[u] = w;
                parent[u] = v;
                PQ.push({-d[u], u}); // push a better d[] of u
            }
        }
    }
    int cost=0, cnt=0;
    for (int i=0; i<n; i++) {
        //fprintf(stderr,"%d",d[i]); //fprintf(stderr,"\n");
        if (d[i]<oo) {
            cost+=d[i];
        } else cnt++;
    }
    if (cnt) printf("-1\n");
    else printf("%d\n", cost);
    return 0;
}

```

以下是 Kruskal 演算法的範例程式。因為要排序，所以我們定義了一個 struct 與一個比較函數用來存邊以及做排序。其他的部分就是根據演算法，對每一根邊檢查兩端是否是同一個區塊，若否，則加入該邊。

```
// Kruskal algorithm O(mlogn)
#include<bits/stdc++.h>
using namespace std;
#define N 10010
#define oo 1000000001
struct EDGE {
    int u,v,w;
};
bool cmp(EDGE &p, EDGE &q) {
    return p.w < q.w;
}
int p[N]; // parent
int rfind(int v) {
    if (p[v]<0) return v;
    return p[v]=rfind(p[v]);
}
// union two set, return true if really merge
bool joint(int u, int v) {
    int r1=rfind(u), r2=rfind(v);
    if (r1==r2) return false;
    if (p[r1]<p[r2]) {
        p[r1] += p[r2];
        p[r2] = r1;
    } else {
        p[r2] += p[r1];
        p[r1] = r2;
    }
    return true;
}

int main() {
    int n, m;
    // input data
    scanf("%d%d", &n, &m);
    vector<EDGE> edge;
    for(int i=0;i<m;i++) {
        int u,v,t;
        scanf("%d%d%d", &u, &v, &t);
        edge.push_back({u,v,t});
    }
    // initial
    for (int i=0;i<n;i++) p[i]=-1; // each vertex a set
    sort(edge.begin(), edge.end(), cmp); // sort by weight
    int cost=0, n_edge=0;
    for (EDGE e: edge) {
        if (joint(e.u, e.v)) { // insert an edge
            cost += e.w;
            n_edge++;
        }
    }
}
```

```
}  
if (n_edge < n-1) printf("-1\n");  
else printf("%d\n", cost);  
return 0;  
}
```

補充說明：不管邊是正的或是負的，最小生成樹的找法都是一樣，此外，邊的總和最大的最大生成樹以及最大的邊最小的生成樹都可以用類似的方法計算。

Bangye.Wu

8. 樹上演算法

「樹上的問題大部分都可以用 DP 解決，一個變數不夠，就用兩個。」

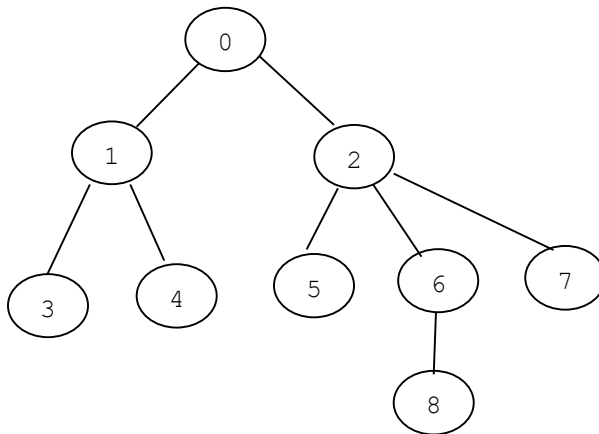
「好想抖著拖鞋在樹上打扣，從上往下看，好多問題都變簡單了。」

這一章介紹樹 (tree)，樹既然是一種圖，圖上的演算法例如 BFS 與 DFS 當然也都能用在樹上。但樹是特別稀疏的圖又有特殊的性質，所以很多問題在樹上有更好的解。另一方面，許多資料結構是使用樹的觀念建構出來的，本章探討的主要是輸入資料是一個樹的問題，而並非用樹做為資料結構的問題，這兩類問題並不相同。樹上的問題大部分都屬於貪心法則或是動態規劃，但需要搭配樹的結構來處理。

8.1. 基本觀念與名詞

回顧一下圖的基本定義，一個圖寫作 $G=(V, E)$ ，表示一個圖名字是 G ，由點的集合 V 與邊的集合 E 組成，其中 V 是點集合，也就是要研究的對象；而 E 是邊的集合，也就是誰與誰之間存在這種關係。樹 (Tree) 是一個連通而無環路的圖，通常指的是無向的簡單圖，也就是說若一棵樹 $T=(V, E)$ ，則 $m=n-1$ ，其中 n 與 m 是點數與邊數。

為了方便或需要，我們可以指定其中一點為根 (root)，稱為有根樹 (rooted tree)，一旦指定根之後，我們可以給圖上的邊訂定方向，每個邊的方向是由根往其他點，或者尤其它點往根的方向。通常把樹畫成一個家族圖，根在最上方，也就是說所有的邊都是一個上下兩點的關係，tree 所代表的二元關係就被比喻為 parent 與 child 的關係。下圖是一個有根樹，其中 0 是根。除了根以外，每個點的上方恰有一點與其相連，這個點稱為他的 parent，例如 4 與 3 的 parent 是 1；而 1 與 2 的 parent 是 0 (根)。若 a 是 b 的 parent，則稱 b 是 a 的孩子 (child)。只要指出每一個點的 parent 就可以惟一決定一棵樹。以下介紹一些基本的知識與名詞。

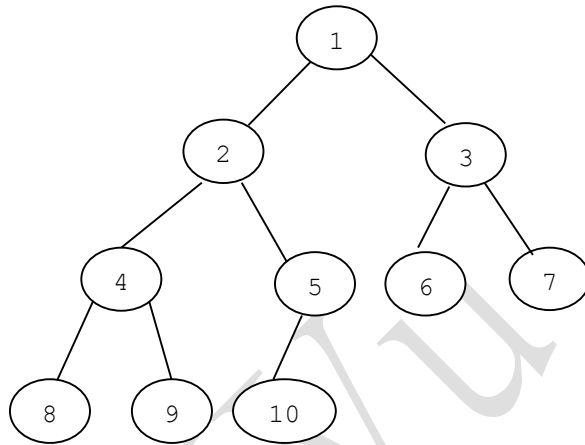


A rooted tree with 9 vertices and 8

- 樹的等價定義 (定理)：樹的定義是「連通無環路的圖」，以下的敘述也都可以證明所代表的是一棵樹。
 - 一個連通圖且 $n=m+1$ ；
 - 一個無環路的圖且 $n=m+1$ ；
 - 任兩點恰有一條路徑的圖；
 - 一個圖，點以 $1\sim n$ 編號，邊的集合是 $\{(i, p(i)) : p(i) < i, 2 \leq i \leq n\}$ 。也就是除了 1 號點之外，每個點連一條邊到編號比他小的點。這個圖其實就是個有根樹， $p(i)$ 就是 i 的 parent，每個點的編號都大於 parent 的編號。
- 每一個連通區塊都是一棵樹的圖就稱為森林 (forest)，也就是說一個森林由若干個不相交的樹組成，有 n 個點的森林，它的邊數是 $m=n-k$ ，其中 k 是連通區塊的個數。
- 在有根樹上，沒有孩子的點稱為樹葉 (leaf)，葉子也稱外節點 (external node)，不是葉子的稱為中間節點 (internal node)。每個點沿著 parent 往上一路走，都會走到 root，中間碰到的點都是它的祖先 (ancestor)。對於一個點 v ，所有以 v 為祖先的點，就是 v 的後代 (descendant)， v 與它的所有後代組成的圖就是以 v 為根的子樹 (subtree rooted at v)。在無根樹上，只有一個鄰居的點也稱為葉節點。
- 有根樹的分支度 (degree) 通常指每個點孩子數量的上限，degree 為 2 的樹 (也就是每個點最多兩個孩子) 稱為二元樹 (binary tree)，二元樹中每個節點的兩個孩子稱為左孩子與右孩子，以左 (右) 孩子為根的子樹就稱為左 (右) 子樹。二元

樹在資料結構中扮演重要的腳色，例如二元搜尋樹(Binary search tree)與堆積(heap)，但本章中用到的觀念不多，因此只做簡略的名詞介紹。

- 完整二元樹(Complete binary tree)是指一個二元樹它的葉節點只在最後兩層，而且如果葉節點在倒數第二層的話，一定在最右邊且連續出現，如右圖。完整二元樹如果用由上而下由左至右連續以 $1 \sim n$ 編號，可以用編號來找 parent 與 child，因此可以用陣列儲存而無需用複雜的資料結構。在右圖的例子中，任一點 v 的左孩子是 $2v$ 而右孩子是 $2v+1$ ，如果這兩個孩子編號沒有超過 n ； v 的 parent 是 $v/2$ (整數除法)。堆積是以完整二元樹的概念建構的，常用來製作優先佇列。
- 二元搜尋樹(Binary search tree, BST)的每一個節點可以存一筆資料，每一筆資料有一個鍵值(key)作為排序的依據，BST 的 key 滿足以下規則：對任何一點 v ，在它左子樹中的點，鍵值必定小於它；在它右子樹中的點，鍵值必定大於它。BST 有不同的變化形式，是很重要到的動態資料結構，一個平衡的 BST 通常樹高不超過 $O(\log(n))$ ，STL 中的 set/map 通常以平衡的 BST 建構。



一些其他名詞我們在後面遇到時再說明。因為樹沒有環路，如果從樹上刪掉任意一點，會造成一些獨立的子樹，既然是獨立不相交的，往往就可以獨自求解，然後在該點上合併子問題的解，這事實上就是分治的概念，如果從非遞迴的角度，它就是動態規劃(DP)。實際解題時，為了方便，通常把樹當成有根樹來做，而實作就區分為 top-down (遞迴) 與 bottom-up (非遞迴) 兩種方式。所以說樹上的問題絕大部分都可以用 DP 來解，以下我們會看到很多例子。

8.2. 樹的儲存與基本演算法

在這一節中我們介紹儲存樹的基本資料結構與幾個基本演算法。

8.2.1. 儲存樹的資料結構

樹是一種圖，儲存圖的資料結構當然可以用來存樹，但是樹是非常稀疏的圖，所以用鄰接矩陣是非常不合適的。如果是無根樹，用鄰接串列是合適的方式。在有根樹時，我們

還有兩種儲存方式：存 parent 以及存 child。使用上當然要看輸入給的方式，也要看程式的寫法。基本上三種方式可以互相轉換。

因為 parent 與 child 是相互的關係，如果題目給的是每個點的 parent (除了 root)，要轉成 child 的形式，就讓每一個點把自己放進 parent 的 child[]；如果題目給的是 child，就由每一個 parent 設定自己孩子的 parent。以下是簡單的範例，我們寫了兩個函數，一個從 parent 轉成 child，另外一個從 child 轉成 parent。其中要注意的是，如果 degree 沒有限定，child[] 要使用 vector 來做才比較方便 (因為陣列長度未知)。另外轉到 parent 時需要留意，因為 root 將來是沒有 parent，如果不確定 root 的編號，初設時要將 parent 設成不可能的值 (如 -1)，那麼在轉換後才能找出 root。

```
// tree, data structure demo, parent and child
#include <bits/stdc++.h>
using namespace std;
#define N 1000
int parent[N];
vector<int> child[N]; // list of children

// parent to child
void p2c(int n) {
    for (int i=0; i<n; i++)
        child[i].clear();
    for (int i=0; i<n; i++) // set i as a child of parent[i]
        child[parent[i]].push_back(i);
    return;
}

// child to parent, parent[root] = -1, return root
int c2p(int n) {
    for (int i=0; i<n; i++)
        parent[i] = -1; // -1 as no parent
    for (int i=0; i<n; i++) {
        for (int c: child[i]) // each child of i
            parent[c] = i;
    }
    for (int i=0; i<n; i++) // find root
        if (parent[i]==-1)
            return i;
}
```

child 或 parent 轉成無根樹的连接串列形式非常簡單，因為一個點的鄰居就是 parent 以及所有的 child。另一方面，要將無根樹的连接串列轉成有根樹的 child 或 parent 的形式則須要做一個 BFS 或 DFS 走訪，以下介紹 BFS 與 DFS。

8.2.2. DFS

樹的 DFS 相當重要，大部分樹的題目都可以用 DFS 來解。樹既然是圖的一種，當然可以用圖的 DFS 來做樹的 DFS，但另一方面樹是一種特殊而簡單的圖，所以可以更簡單的來做。有根樹與無根樹的走訪很類似，我們先用一個例題來示範無根樹的 DFS。

P-8-1. 樹上的推銷員

有一個推銷員要走訪 n 個城市並在結束後回到出發的城市。這些城市以 $n-1$ 條道路連接，每條道路連接兩個不同的城市並且雙向可以通行，而且已知每一個城市都可以到達，不會有到達不了的狀況。輸入道路的資料，請你找出一個長度最短的城市走訪順序，因為這樣的順序很多，你必須輸出字典順序最小的那一個。字典順序是用來比較兩個序列的先後順序，其方法是由前往後找到第一個不相同的位置，該元素比較小的序列就是順序比較小，如同在英文字典中單字的順序一般。例如 $(0, 2, 3, 1) < (0, 3, 1, 2)$ ，又 $(0, 3, 2, 1) < (1, 0, 2, 3)$ 。

舉例來說，如右圖，有 5 個城市以 4 條道路連接，假設每條道路的長度都是 1。字典順序最小的最短拜訪順序是 $(0, 1, 0, 4, 2, 4, 3, 4, 0)$ 。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，代表城市的數量，城市是以 $0 \sim n-1$ 編號，接下來有 $n-1$ 行是道路的資料，每行三個整數 a, b 與 w ，代表此道路連接城市 a 與 b ，道路的長度是 w 。 n 不超過 50000，每條道路長度是不超過 100 的正整數。

輸出：第一行輸出最短的旅行距離，第二行輸出拜訪順序，相鄰的數字之間要以一個空白間格。

範例一輸入：	範例一輸出：
5	8
1 0 1	0 1 0 4 2 4 3 4 0
0 4 1	
2 4 1	
4 3 1	

範例二輸入：	範例二輸出：
3	16
1 2 5	0 2 1 2 0
0 2 3	

n 個城市以 $n-1$ 條道路連接而且每個城市都可以彼此到達，顯示是一個 n 個點 $n-1$ 個邊的連通圖，所以它是個樹。旅行推銷員問題在一般圖上是很難解的問題，我們在第 6 章 DP 中有見過，但是在樹上卻相當簡單：最短的環路一定每個邊恰好走兩次。這可以用歸納法很簡單的證明，想像 v 是一個葉節點， P 是走訪其他 $n-1$ 個城市的最短環路，我們只要沿著 P 走，走到 v 的惟一鄰居時，繞過去 v 就行了。

所以任何一個 DFS 走的順序都是最短的環路，這一題的問題其實是要找字典順序的 DFS。在做圖的 DFS 時，有一個重點是對每個點需要設定一個 `visit[]` 值，來避免重複走訪到已經走過的點，但樹的 DFS 可以省略這個動作，取而代之的是遞迴呼叫 v 時，傳過去從哪個點 (p) 呼叫它，因為樹沒有環路，只要不要遞迴呼叫 p ，就不會重複走訪到同一點。請看下面的範例程式：

第 4 行的 `adj[v]` 用來存每個點 v 的鄰居，第 5 行的 `tour` 是用來放答案的環路。第 7~14 行就是 DFS 的遞迴函數。每次走到 v 我們先將 v 放入 `tour` 當做拜訪 v 的動作，接著對它的每個鄰居 u (除了來時路的 p)，遞迴呼叫 `dfs(v,u)` 以拜訪 u 點以下所有點，然後回到 v 。主程式部分，第 17~24 行是輸入，我們把邊長另外加總處理。輸入完畢後，先將每個點的鄰居依照編號排序，然後以 `dfs(-1,0)` 作為以 0 為起點的走訪。最後，第 30~34 行是輸出答案。

```

00 // lexicographic first dfs
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 50010
04 vector<int> adj[N]; // neighbor
05 vector<int> tour;
06 // dfs from v, parent=p
07 void dfs(int p, int v) {
08     tour.push_back(v);
09     for (int u: adj[v]) { // dfs each neighbor
10         if (u==p) continue; // except for parent
11         dfs(v,u);
12         tour.push_back(v);
13     }
14 }
15 int main() {
16     int n, total=0;
17     scanf("%d", &n);

```

```

18     for (int i=0;i<n-1;i++) {
19         int u,v, w;
20         scanf("%d%d%d", &u, &v, &w);
21         adj[u].push_back(v);
22         adj[v].push_back(u);
23         total += w; // total edge weight
24     }
25     for (int i=0; i<n; i++)
26         sort(adj[i].begin(), adj[i].end());
27     dfs(-1,0); // starting at 0, no parent
28     assert(tour.size()==2*n-1);
29     // output
30     printf("%d\n", total*2);
31     printf("%d", tour[0]);
32     for (int i=1; i<tour.size();i++)
33         printf(" %d", tour[i]);
34     printf("\n");
35     return 0;
36 }

```

這一題以無根樹的走訪來做就可以，但是為了示範有根樹的 DFS，我們也寫一個將無根樹轉換為有根樹之後再做 DFS 的範例程式，請看以下程式。

誠如上面所說，這個程式基本上是 DFS 走兩次。第 8~16 行的 `root(p,v)` 是做將樹以 `v` 點為 `root` 吊起來的動作，除了 `p` 以外的所有鄰居都變成 `v` 的 `child`。並且對每一個 `v` 的孩子 `u`，遞迴呼叫 `root(v,u)`，這樣完成了無根樹轉成有根樹的動作。

然後在第 35 行將各點的 `child` 排序後，呼叫 `tsp(0)` 來進行以 0 點開始的 DFS 走訪。因為已經轉成有根樹，遞迴走訪的函數 `tsp(v)` 不需要指名 `parent` 是誰，先拜訪 `v` 點後對每一個孩子 `ch`，進行遞迴拜訪 `tsp(ch)` 以及回到 `v` 的動作。

```

00 // lexicographic first dfs. transform to rooted tree
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 50010
04 vector<int> adj[N]; // neighbor
05 vector<int> child[N];
06 vector<int> tour;
07 // rooting tree at from v, parent=p
08 void root(int p, int v) {
09     // except for parent, all neighbors are children
10     for (int u: adj[v]) {
11         if (u!=p) {
12             child[v].push_back(u);
13             root(v,u);
14         }
15     }
16 }

```

```

17 void tsp(int v) {
18     tour.push_back(v);
19     for (int ch: child[v]) { // for each branch
20         tsp(ch);
21         tour.push_back(v);
22     }
23 }
24 int main() {
25     int n, total=0;
26     scanf("%d", &n);
27     for (int i=0; i<n-1; i++) {
28         int u, v, w;
29         scanf("%d%d%d", &u, &v, &w);
30         adj[u].push_back(v);
31         adj[v].push_back(u);
32         total+=w;
33     }
34     root(-1, 0);
35     for (int i=0; i<n; i++) // sort children
36         sort(child[i].begin(), child[i].end());
37     tsp(0);
38     assert(tour.size()==2*n-1);
39     printf("%d\n", total*2);
40     printf("%d", tour[0]);
41     for (int i=1; i<tour.size(); i++)
42         printf(" %d", tour[i]);
43     printf("\n");
44     return 0;
45 }

```

這是本章第一個樹的 DFS，請務必了解，因為樹的問題大部分都是利用 DFS 的架構去做的。

8.2.3. BFS

樹上的 BFS 與 DFS 一樣的都是由起點往外走，如果把起點當 root，那麼兩者走出來的順序固然不同，但都是一個 top-down 的順序，也就是任何一點都在它的孩子之前被拜訪，如果把這個順序反過來，就是一個 Bottom-up 的順序。樹的問題有很多都是 DP 或貪心法則，而都需要一個 top-down 或 bottom-up 的順序，所以通常用 DFS 的比較多，原因是 DFS 比較好寫。

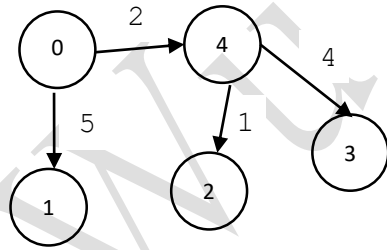
圖的 BFS 與 DFS 我們已經看過了，BFS 需要使用一個 queue，而 DFS 只要遞迴就好了，上一節我們也看到樹的 DFS 還更簡單一點，競賽與考試的時候，程式簡潔好寫是一個極大的優點，所以寫 DFS 的比較多。在圖的時候，BFS 還有它的必要，例如在無加權圖上求起點到個點距離，這個理由在樹上也不見了，因為樹沒有環路，不管是加權圖或

是非加權圖，都可以用 DFS 來計算起點到個點的距離。結論是樹上用 BFS 的機會比較少，如果遞迴可用，通常用 DFS；只有因為某些因素遞迴不可用（例如環境限制遞迴深度），而需要 top-down 順序時才會使用。

我們以起點到各點的距離來展示樹上的 BFS。

P-8-2. 物流派送系統

有一個物流派送系統，系統有 n 個工作站與 $n-1$ 條輸送帶，工作站以 $0 \sim n-1$ 編號，其中 0 號站是進貨站，所有物品都是由 0 號站進入系統然後經過輸送帶配送到某個站。每個輸送帶都是單向的從某個站 a 把物品送到站 b ，輸送的時間是 $w(a, b)$ 。已知由進貨站可以經由輸送帶直接或間接轉送到達任何其他站，請計算下列兩者 (1). 由進貨站到達運送時間最長的站需要多少時間，以及 (2). 由進貨站需要最多次轉運的需要經過幾次輸送帶。



舉例來說，如右圖，有 5 個站以 4 條輸送帶連接，假設每條輸送帶的運送時間如圖上標示。需要最長運送的時間是 3 號站 ($2+4=6$)，最多需要經過的輸送帶是兩次 (2 號站與 3 號站)。

Time limit: 1 秒

輸入格式：第一行是正整數 n ， $n > 1$ ，接下來有 $n-1$ 行是輸送帶的資料，第 i 行 2 個整數 x 與 w ，代表此輸送帶連接 x 與 i ，輸送的時間是 w 。 n 不超過 $1e5$ ，每條輸送帶的輸送時間是不超過 1000 的正整數。

輸出：第一行輸出最長的運送總時間，第二行輸出最多經過幾次輸送帶。第一行與第二行的答案不一定是同一個站，除了輸送帶以外的時間均忽略不記。

範例一輸入： 5 0 5 4 1 4 4 0 2	範例一輸出： 6 2
範例二輸入： 6 0 9 0 1 2 1 5 3 3 2	範例二輸出： 9 4

範例一：題目中的例子

範例二：時間最久的是 0->1，時間是 9；最多次的是 0->2->3->5->4，共四次。
(測資 1 與 2 的 $n \leq 100$ 且前一站的編號必然小於該站。)

工作站是點，輸送帶是邊， n 個點 $n-1$ 個邊，而 0 可以到達所有的點，這個圖形就是一個有根樹，邊有長度，第一個要求的是根到各點的最大距離，第二個是經過最多次的邊，那也就是無加權圖的最大距離。輸入給的形式是每個點的前一站，也就是 parent。

在樹上計算根(0)到所有 v 的距離其實不過就是個簡單的 DP，遞迴式

$$d[v] = d[\text{parent}[v]] + w(\text{parent}[v], v) \text{ for all } v > 0$$

所以只要找出所有點都出現在它 parent 之後的 top-down 序列，然後根據遞迴式計算即可，以下是這樣寫的範例程式。在輸入的時候，除了 parent 之外，我們也轉存每個點的 child(第 14 行)，各點 i 到它 parent 之間的邊長則存在 $\text{leng}[i]$ 。第 17 行宣告一個 `vector<int>` 當作 queue 來使用，然後開始一個 BFS 走訪，因為是 tree，我們不需要記錄該點是否已拜訪過，所以 while 迴圈中只是簡單的從 queue 中取出一點，然後將它的孩子都放進 queue 中。接著第 28~33 行計算最遠的加權距離，只要沿著順序套遞迴式計算就可以了，最後是無加權圖的距離，我們放在 $\text{step}[v]$ ，它的算法相同，只是每個邊都算 1 罷了。

```

00 // longest distance d(0,v), BFS, two pass
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 100010
04 vector<int> child[N]; // child of i
05 int parent[N], leng[N];
06 int main() {
07     int n;
08     scanf("%d", &n);
09     for (int i=1; i<n; i++) {
10         int v, w;
11         scanf("%d%d", &v, &w);
12         parent[i]=v;
13         leng[i] = w; // length of edge (i, parent[i])
14         child[v].push_back(i);
15     }
16     // find a top-down sequence
17     vector<int> que;
18     int head=0; // a queue
19     que.push_back(0);
20     while (head < que.size()) {
21         int v=que[head++];
22         for (int ch: child[v])
23             que.push_back(ch);
24     }
25     assert(que.size()==n); // otherwise not a tree
26     // compute distance in top-down sequence
27     int d[N];
28     d[0]=0;
29     for (int i=1; i<n; i++) {
30         int v = que[i];
31         d[v] = d[parent[v]] + leng[v];
32     }
33     printf("%d\n", *max_element(d, d+n));
34     // compute max step, using range-based for
35     for (int v: que) {
36         if (v>0) // 0 has no parent
37             d[v] = d[parent[v]] + 1;
38     }
39     printf("%d\n", *max_element(d, d+n));
40     return 0;
41 }

```

其實找順序與計算距離可以在同一回合做，就像標準的 BFS 一樣。以下就是這樣的寫法。

```

// longest distance d(0,v), BFS, one pass
#include <bits/stdc++.h>
using namespace std;
#define N 100010
vector<int> child[N]; // child of i

```

```

int parent[N], leng[N];
int main() {
    int n;
    scanf("%d", &n);
    for (int i=1; i<n; i++) {
        int v, w;
        scanf("%d%d", &v, &w);
        parent[i]=v;
        leng[i] = w; // length of edge (i, parent[i])
        child[v].push_back(i);
    }
    // find a top-down sequence
    vector<int> que; // a queue
    int d[N], step[N], head=0, longest=0, max_step=0;
    d[0]=0, step[0]=0;
    que.push_back(0);
    while (head < que.size()) {
        int v=que[head++];
        for (int ch: child[v]) {
            que.push_back(ch);
            d[ch] = d[v] + leng[ch];
            longest = max(longest, d[ch]);
            step[ch] = step[v] + 1;
            max_step = max(max_step, step[ch]);
        }
    }
    assert(que.size()==n); // otherwise not a tree
    printf("%d\n%d\n", longest, max_step);
    return 0;
}

```

如同前面說的，tree 上找距離也可以用 DFS，DFS 用遞迴也不需要 queue，所以確實更簡潔。

```

// longest distance d(0,v), DFS
#include <bits/stdc++.h>
using namespace std;
#define N 100010
vector<int> child[N]; // child of i
int leng[N]; // length to its parent
int d[N], step[N]; // distance and number of step to v
int longest=0, max_step=0; // answer
void dfs(int v) {
    for (int ch: child[v]) {
        d[ch] = d[v] + leng[ch];
        longest = max(longest, d[ch]);
        step[ch] = step[v] + 1;
        max_step = max(max_step, step[ch]);
        dfs(ch);
    }
    return;
}

```

```

}

int main() {
    int n;
    scanf("%d", &n);
    for (int i=1; i<n; i++) {
        int v, w;
        scanf("%d%d", &v, &w);
        leng[i] = w; // length of edge (i, parent[i])
        child[v].push_back(i);
    }
    d[0]=0, step[0]=0;
    dfs(0); // DFS from root 0
    printf("%d\n%d\n", longest, max_step);
    return 0;
}

```

8.2.4. 由下而上的走訪

由下而上計算是很多樹的演算法所需要的順序，如果遞迴可以用，通常優先使用 DFS，如果不行，就必須自己找一個 Bottom-up 順序。非遞迴的找此順序可以採用上一節的 BFS，因為將 top-down 順序反轉過來就是 bottom-up 順序。但這樣需要先找順序再做計算，通常要兩回合。有時我們希望一個回合直接一面找順序一面做計算，我們在 P-3-1 已經介紹過這樣的做法了，其實就是把樹看成一個由下而上的 DAG。我們準備一個 queue 放置已經可以出列的點，一開始把所有葉節點都放進 queue，每一回合從 queue 中取出一點，將它的 parent 的孩子數減一，若它的 parent 變成葉子，就放進 queue 中。

我們以下一個題目來展示 bottom-up 序列的計算與運用。

P-8-3. 購物中心選位置

有 n 個小村莊以 $0 \sim n-1$ 編號，其中編號 0 的是市政府所在地，這些村莊以 $n-1$ 條道路連接，每條道路都是雙向通行且連接兩個不同的村莊，已知從任一個村莊 i 都可以經由這些道路到達市政府所在的 0 號村莊，所以也可以到達任何其他村莊，而且由村莊 i 往市政府會經過的第一站村莊是 $p(i)$ 。現在某大集團希望選其中一個村莊來蓋一個購物中心，選址的判斷方式是購物中心到所有村莊的距離總和要越小越好，請你幫忙計算哪一個村莊是最好的位置，到各村莊的距離總和最小是多少。

Time limit: 1 秒

輸入格式：第一行是正整數 n ， $n > 1$ ，村莊以 $0 \sim n-1$ 編號。接下來有 $n-1$ 行是道路的資料， i 由 1 到 $n-1$ ，第 i 行有 2 個整數 $p(i)$ 與 $w(i)$ ，代表 i 往 0 號村莊的道路第一個會經過 $p(i)$ ，而 i 與 $p(i)$ 之間的道路長度是 $w(i)$ 。 n 不超過 $1e5$ ，每條道路的長度是不超過 1000 的正整數。

輸出：第一行輸出最佳的村莊編號，如果有超過一個村莊都是最佳位置，則找離市政府最遠的那一個，第二行輸出最佳村莊到各村莊距離總和。

範例一輸入：

```
5
0 5
4 1
4 4
0 2
```

範例一輸出：

```
4
14
```

範例二輸入：

```
6
0 9
0 1
2 1
5 3
3 2
```

範例二輸出：

```
3
21
```

範例一說明：最佳位置是 4 號村莊，4 到其它村莊的距離總和是 14。

範例二說明：2 與 3 到其它的距離總和都是最小的 21，而 3 離 0 的距離比 2 與 0 的距離遠。

這些村莊與道路構成的圖顯然是個樹， $p(i)$ 就是 i 的 parent，0 號點是 root。令 $D(v)$ 是點 v 到所有其他點的距離總和，對任一點 v ，我們可以在 $O(n)$ 的時間計算出 $D(v)$ ，所以對每一點 v 都求出 $D(v)$ 來再找最小值的方法可以在 $O(n^2)$ 完成，但是還是不夠快，因為本題要求的 n 是 $1e5$ 。

假設這些點是在一條數線上，也就是 n 個數字要找其中一個與其它數字的距離總和最小，那麼答案是這 n 個數字的中位數 (median)，也就是排序後位置在中間的那個數字，如果 n 是偶數，則中間那兩個都是解 (到各點距離相同)。這個證明其實很簡單，對於任意一點，如果他的左右有一邊的點數超過 $n/2$ ，例如左邊有 x 個點， $x > n/2$ ，則將點

往左邊移動長度 y ，左方 x 個點距離減少 xy ，右邊 $(n-x)$ 個點距離增加 $(n-x)y$ ，總距離一定會減少。所以所選的點左右兩邊的點數都不會超過 $n/2$ 。

在圖上與其它點距離總和最小的點也是稱為 median。樹的 median 與數字的 median 有一樣的性質，事實上數線上的點就是樹的退化情形。若 v 是 $D(v)$ 最小的 v 點，則 v 的每一個分枝的點數都不會超過 $n/2$ 。這一題的輸入給的方式是以 0 為 root 的有根樹，因此找 median 其實是要找子樹的點數。令 $\text{num}(v)$ 是以點 v 為根的子樹的點數，也就是 v 的後代總數（含 v 自己），我們要找的是“最低”的某個點 s ，滿足 $\text{num}(s) \geq n/2$ ，最低的意思是，對所有 s 的孩子 c ， $\text{num}(c) < n/2$ 。如果 $\text{num}(s)$ 恰好是 $n/2$ ，則 s 與其 parent 都是 median，根據本題的題意，此時要輸出比較低的那一個 median。請注意，數學上「 $\geq n/2$ 」在 C 程式中必須寫成「 $\geq (n+1)/2$ 」，因為整數除法捨位的關係。

所以本題最主要的是算各子樹的大小 $\text{num}(v)$ ，算完之後，可以依照定義找某個點，它的孩子的子樹都不到 $n/2$ 而它的子樹不小於 $n/2$ ，最後再計算到其他點距離。至於計算各子樹的大小的方法，以下是一種想法：因為每個點都是它每個祖先的後代，沿著 parent 一路往上走碰到的都是祖先，所以，我們可以對每個點都把他加到它所有的祖先中。以下是這樣寫出來的程式，主程式一開始先做輸入，第 25~32 是算 $\text{num}[]$ ；第 34~41 行是依照定義找 median；最後用 DFS 來計算距離。

```

00 // P-8-3 tree median, O(nh) method
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 100010
04 vector<pair<int,int>> adj[N]; // (neighbor, length) of i
05 int parent[N], d[N]={0};
06 // compute distance
07 void dfs(int v, int p) {
08     for (auto e: adj[v]) {
09         if (e.first != p) {
10             d[e.first] = d[v] + e.second;
11             dfs(e.first, v);
12         }
13     }
14 }
15 int main() {
16     int n;
17     scanf("%d", &n);
18     for (int i=1; i<n; i++) {
19         int w;
20         scanf("%d%d", &parent[i], &w);
21         adj[i].push_back({parent[i], w});
22         adj[parent[i]].push_back({i, w});
23     }
24     parent[0]=-1; // root has no parent
25     int num[N]={0}; // number of node in the subtree

```

```

26     for (int i=0; i<n; i++) { // for each vertex
27         int v=i;
28         while (v>=0) { // add itself to all its ancestors
29             num[v]++;
30             v=parent[v];
31         }
32     }
33     // find median
34     bool flag[N]={false}; // if having a child >=n/2;
35     for (int i=1; i<n; i++)
36         if (num[i] >= (n+1)/2) flag[parent[i]]=true;
37     int median=0;
38     for ( ; median<n; median++) {
39         if (num[median] >= (n+1)/2 && !flag[median])
40             break;
41     }
42     // compute d[v] the distance from median to v
43     int total=0;
44     dfs(median, -1);
45     for (int i=0; i<n; i++)
46         total += d[i];
47     printf("%d\n%d\n", median, total);
48     return 0;
49 }

```

這個程式的主要缺點是算得太慢，複雜度太差，如果樹的高度是 h ，需要的時間是 $O(nh)$ ，所以在樹高是 $O(n)$ 時，它的時間複雜度是 $O(n^2)$ 。其實，根據定義： $num(v) = 1 + \sum_{u \in CHILD(v)} num(u)$ ，因此計算 $num(v)$ 不過就是個類似於前綴和的簡單 DP，只要走一個 bottom-up 的順序，當一個點的子樹大小算好了之後，把它加到 parent 的子樹大小中就可以了。這樣做足夠改善複雜度，但是程式還是比較複雜囉嗦，子樹大小都算好了，是不是可以用來計算第三步驟要算的距離呢？

若 m 是 median，我們要算 m 到所有點的距離和，直接的方法當然是把每一條路徑都算出來再加總，其實在樹上有另外一種計算方式。考慮任一條邊 (u, v) ，其中 v 是 u 的 parent，邊 (u, v) 把樹分成兩塊，姑且稱為下方與上方，點 m 在 (u, v) 的某一方，那麼，哪些點到 m 的路徑會經過 (u, v) 呢？當然是 m 另外一方的點才需要跨過此邊，所以我們只要知道另外一方有幾點，就知道 (u, v) 在距離總和中會被計算多少次，對每個邊都這樣計算後加總就是距離總和了。那麼， m 在哪一方？根據 median 的性質， m 一定在點多的那一方！！我們在計算 $num[]$ 時算出了每一個子樹的大小，所以我們知道 $num[u]$ ，不管 m 在上方或下方，這條邊被計算的次數是 $\min(num[u], n - num[u])$ 。有了這個了解，我們就可以直接算距離總和了。以下是用 bottom-up 順序做的範例程式。

輸入的時候順便計算每個點的孩子數量 $deg[]$ ，第 15 行準備一個佇列 Q 放可以出列的點，初始時把葉節點 ($deg[] = 0$) 放入 Q 中，然後跑一個 while 迴圈做 bottom-up 走

訪。每次迴圈從佇列中拿出一點 v 來，因為它一定在它的所有孩子之後出列，所以它的孩子們的子樹大小都已經加到它的 $\text{num}[v]$ 中了，因為我們尚未算 v 自己，所以第 22 行將 $\text{num}[v]++$ ，第 23 行檢查它是否是第一個超過 $n/2$ 的子樹，初始我們將 median 設為 -1 ，所以第 23 行的 if 成立時， v 就是最低的 (也就是我們要找的) median 。第 25 行我們計算到 median 的距離總和中這個邊會被計入的量，接著，將它的 $\text{num}[v]$ 加入 parent 的子樹大小中，並檢查 parent 是否要進入 Q 中。這個程式要簡潔多了，時間複雜度是 $O(n)$ 。

```

00 // P-8-3 tree median, Bottom-up method, O(n)
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 100010
04
05 int main() {
06     int n, parent[N], leng[N];
07     int deg[N]={0}, num[N]={0};
08     scanf("%d", &n);
09     for (int i=1; i<n; i++) {
10         scanf("%d%d", &parent[i], &leng[i]);
11         deg[parent[i]]++;
12     }
13     parent[0]=-1; // root has no parent
14     int median=-1, total=0;
15     queue<int> Q;
16     for (int i=0; i<n; i++)
17         if (deg[i]==0) Q.push(i);
18     // bottom-up traversal
19     while (!Q.empty()) {
20         int v=Q.front();
21         Q.pop();
22         num[v]++; // add itself
23         if (median<0 && num[v]>=(n+1)/2)
24             median = v; // first >=n/2 is median
25         total += min(num[v], n-num[v]) * leng[v];
26         // check parent
27         if (parent[v]<0) break; // root
28         num[parent[v]] += num[v];
29         if (--deg[parent[v]] == 0)
30             Q.push(parent[v]);
31     }
32     printf("%d\n%d\n", median, total);
33     return 0;
34 }

```

如果用遞迴當然程式碼更簡潔一點，遞迴函數中，記得要先呼叫孩子後，才能把它的值拿來使用。

```

// P-8-3 tree median, dfs method, O(n)
#include <bits/stdc++.h>
using namespace std;
#define N 100010
vector<pair<int,int>> child[N];
int n, num[N]={0}, median=-1, total=0;
void dfs(int v) {
    for (auto e: child[v]) {
        int u=e.first, w=e.second;
        dfs(u);
        total += min(num[u], n-num[u]) * w;
        num[v] += num[u];
    }
    num[v]++; // add itself
    if (median<0 && num[v]>=(n+1)/2)
        median = v;
    return;
}

int main() {
    scanf("%d", &n);
    for (int i=1;i<n;i++) {
        int v, w;
        scanf("%d%d", &v, &w);
        child[v].push_back({i,w});
    }
    dfs(0);
    printf("%d\n%d\n", median, total);
    return 0;
}

```

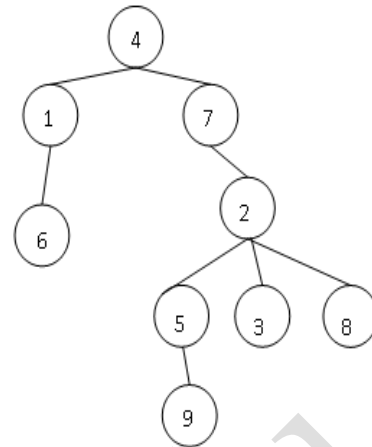
8.3. 例題與習題

在這一節中我們介紹一些典型的樹狀圖的題目，不外乎貪心法或遞迴，也有些是模擬或走訪。下一題曾經出現在 P-3-1，當時是介紹由下而上的走訪方式，而且還沒介紹什麼是 DP，在學過 DP 以及樹的 DFS 之後，這一題應該變得簡單很多了。

P-8-4. 樹的高度與根 (同 P-3-1) (APCS201710)

有一個大家族有 N 個成員，編號 $1 \sim N$ ，我們請每個成員寫下此家族中哪幾位是他的孩子。我們要計算每個人有幾代的子孫，這個值稱為他在這個族譜中的高度，也就是說，編號 i 的人如果高度記做 $h(i)$ ，那麼 $h(i)$ 就表示在所有 i 的子孫中，最遠的子孫與他隔了幾代。本題假設除了最高的一位祖先 (稱為 root) 之外，其他成員的 parent 都在此家族中 (且只有一個 parent)。本題要計算所有成員高度的總和以及根的編號。

右圖是一個例子，每個成員是一個圈起來的號碼，劃線相聯的表示上面的點是下面點的 parent。編號 4 是根，有兩個孩子 1 與 7。編號 6, 9, 3, 8 都沒有孩子， $h(6)=h(9)=h(3)=h(8)=0$ ，此外 $h(2)=2$ 因為 9 與他隔兩代。你可以看出來 $h(5)=h(1)=1$ ， $h(7)=3$ ， $h(4)=4$ ，所以高度總和是 11。



Time limit: 1 秒

輸入格式：第一行有一個正整數 N 。接下

來有 N 行，第 i 行的第一個數字 k 代表 i 有 k 個孩子，第 i 行接下來的 k 個數字就是孩子的編號。每一行的相鄰數字間以空白隔開。 $N \leq 1e5$ 。

輸出：第一行輸出根的編號，第二行輸出高度總和。

範例輸入：

```

9
1 6
3 5 3 8
0
2 1 7
1 9
0
1 2
0
0

```

範例結果：

```

4
11

```

因為一個點的高度，是他所有孩子最高的高度加一，這是一個遞迴定義，使用 DFS 來做就非常簡單，對每一個孩子呼叫一次高度計算，然後在所有孩子中取最大值就可以了。

```

// tree height, dfs version, using vector
#include <bits/stdc++.h>
using namespace std;
#define N 100010
int height[N]; // height of node i
vector<int> child[N]; // children of node i

```

```

// recursively find height of r
void findh(int r) {
    height[r] = 0;
    for (int v: child[r]) {
        findh(v); // recursive call
        height[r] = max(height[r], height[v]+1);
    }
}

int main() {
    int i, j, n, root;
    int parent[N]={0}; // parent of node i
    scanf("%d", &n);
    for (i=1; i<=n; i++) {
        int deg, ch;
        scanf("%d", &deg);
        for (j=0; j<deg; j++) {
            scanf("%d", &ch);
            child[i].push_back(ch);
            parent[ch] = i;
        }
    }
    // find root, the node without parent
    for (root=1; root<=n; root++)
        if (parent[root] == 0) break;
    findh(root);
    int total_h = 0;
    for (i=1; i<=n; i++) total_h += height[i];
    printf("%d\n%d\n", root, total_h);
    return 0;
}

```

如果只做純遞迴，時間複雜度會變成 $O(nh)$ ，其中 h 是樹的高度，下面是一個寫壞的方式。

```

// tree height, puring recursion,  $O(nh)$ 
#include <bits/stdc++.h>
using namespace std;
#define N 100010
//int height[N]; // height of node i
vector<int> child[N]; // children of node i
// recursively find height of r
int findh(int r) {
    int h=0;
    for (int v: child[r])
        h = max(h, findh(v)+1); // recursive call
    return h;
}

int main() {
    int i, j, n, root;

```

```

int parent[N]={0}; // parent of node i
scanf("%d", &n);
for (i=1; i<=n; i++) {
    int deg, ch;
    scanf("%d", &deg);
    for (j=0; j<deg; j++) {
        scanf("%d", &ch);
        child[i].push_back(ch);
        parent[ch] = i;
    }
}
// find root, the node without parent
for (root=1; root<=n; root++)
    if (parent[root] == 0) break;
int total_h = 0;
for (i=1; i<=n; i++)
    total_h += findh(i);
printf("%d\n%d\n", root, total_h);
return 0;
}

```

純遞迴的時間複雜度會差，是因為我們沒有把做過的事情用表格記錄下來，如果用 top-down DP 的觀念，使用表格紀錄，即使對每個點呼叫一次，時間複雜度依然只有 $O(n)$ 。下面是不懂樹但是懂 top-down DP 的寫法。

```

// tree height, top-down DP, O(n)
#include <bits/stdc++.h>
using namespace std;
#define N 100010
int height[N]; // height of node i
vector<int> child[N]; // children of node i
// top-down memo DP to find height of r
int findh(int r) {
    if (height[r]>=0) return height[r];
    height[r]=0;
    for (int v: child[r])
        height[r] = max(height[r], findh(v)+1); // recursive call
    return height[r];
}

int main() {
    int i, j, n, root;
    int parent[N]={0}; // parent of node i
    scanf("%d", &n);
    for (i=1; i<=n; i++) {
        int deg, ch;
        scanf("%d", &deg);
        for (j=0; j<deg; j++) {
            scanf("%d", &ch);
            child[i].push_back(ch);
        }
    }
}

```

```

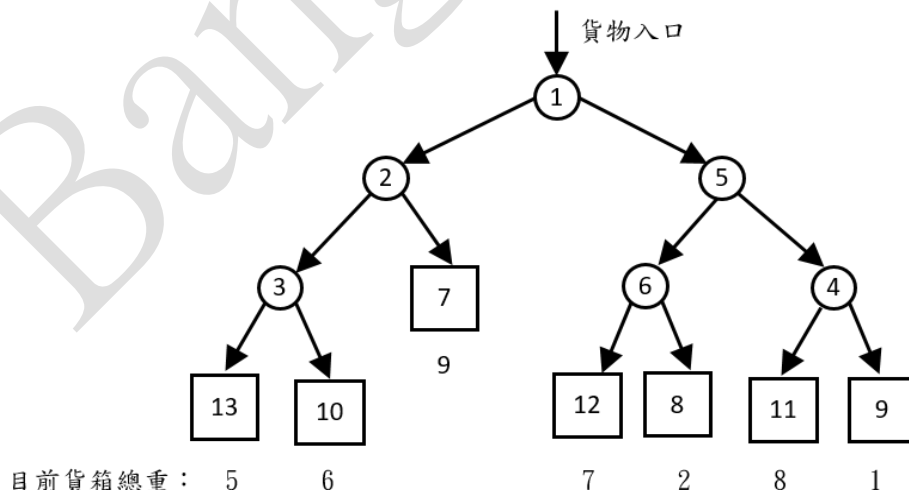
    parent[ch] = i;
}
}
// find root, the node without parent
for (root=1; root<=n; root++)
    if (parent[root] == 0) break;
int total_h = 0;
for (int i=1; i<=n; i++)
    height[i]=-1; // not computed
for (i=1; i<=n; i++)
    total_h += findh(i);
printf("%d\n%d\n", root, total_h);
return 0;
}

```

就解題的目的，DFS 版本最簡單，一個問題我們提出了多個寫法，目的是讓讀者了解其中的道理。下一個問題也出自 APCS 的考試中。

P-8-5. 自動分裝 (APCS202002)

某工程公司設計了一個自動分裝的系統。貨物會一個一個的被送進此系統，經過一些切換器的轉送後，會被輸送到 n 個貨箱。系統有 $n-1$ 個切換器與 n 個貨箱。每個切換器有一個入口以及左右兩個出口，切換器的出口會接到其他切換器或是連接到貨箱，貨箱則只有入口。下圖是一個 $n=7$ 的例子，圓代表切換器，方形代表貨箱，請注意編號 $1 \sim n-1$ 的裝置是切換器，貨箱編號是 $n \sim 2n-1$ ，且貨物入口一定是 1 號切換器的入口。



每一個切換器會分別記錄左右兩個出口所通往貨箱的總重量，當貨物進入此切換器時，切換器會將貨物轉送到「貨箱總重量比較輕的那個出口」，如果兩邊一樣重，則送往左邊。以上圖的例子來說，假設每一個貨箱目前的重量如各矩形下方的標示，下一個到達的貨物的運送過程如下：一號切換器左邊出口是連到 {13, 10, 7} 三個

貨箱，目前總重 $5+6+9=20$ ；右邊出口連到的是 $\{12, 8, 11, 9\}$ 四個貨箱，目前總重 $7+2+8+1=18$ ，因此貨物會由右邊出口送到 5 號切換器。5 號切換器的左邊與右邊的貨箱總重是一樣的 ($7+2=8+1$)，因此貨物由左出口送至 6 號切換器。最後，6 號切換器將貨物送到 8 號貨箱 ($7>2$)。貨物進入貨箱後就存放在該貨箱，該貨物的重量就會加入該貨箱的總重，因此可能影響下一個貨物的運送路徑。

輸入此系統的連接架構與貨箱目前的重量，以及接下來依序進入的 m 個貨物的重量，請計算這 m 個貨物分別會被送到哪一個貨箱。

Time limit: 1 秒

輸入格式：第一行為兩個正整數 n 和 m ，其中 $n \leq 1e5$ 且 $m \leq 100$ 。第二行有 n 個非負整數，依序是編號為 $n \sim 2n-1$ 各個貨箱初始的重量。第三行是 m 個正整數，代表接下來依序進入的貨物重量。全部貨箱初始的重量與貨物重量總和不會超過 10^9 。第四行開始有 $n-1$ 行，這些是系統架構的資訊：每一行有三個整數 p, s 與 t ，代表裝置 p 的左右出口分別接到裝置 s 與 t ，其中 p 一定是一個切換器的編號 ($1 \leq p < n$)。同一行數字之間以空白間隔。

輸出：輸出一行有 m 個整數，依序代表接下來進入系統的 m 個貨物所進入到的貨箱編號，數字之間以一個空白間隔。

範例一：輸入 <pre>4 5 0 0 0 0 5 3 4 2 1 1 2 3 2 4 5 3 6 7</pre>	範例一：正確輸出 <pre>4 6 7 5 5</pre>
範例二：輸入 <pre>7 2 9 2 1 6 8 7 5 2 3 1 2 5 2 3 7 3 13 10 4 11 9 6 12 8 5 6 4</pre>	範例二：正確輸出 <pre>8 7</pre>

範例二的架構即是題目中的圖。

本題所描述的系統其實是一個所謂的二元樹(binary tree)的結構，每一個切換器是一個中間節點(internal node)，每一個貨箱是一個葉節點(leaf node)。題目的要求是模擬每一個貨物進入後的流程，要知道貨物在每個切換器會流向左方或右方，我們需要知道左邊與右邊貨箱**總重量**。假設知道每個切換器左右的總重量之後，我們就可以依照題意模擬貨物流向，而且根據流向來更新左右總重，所以在下一個貨物進入時，就可以繼續模擬的動作。

所以解這個題目需要會做 3 件事：(1). 儲存此樹狀結構；(2). 會計算初始每個中間節點左右的總重量；(3) 模擬流程更新重量紀錄。這一題是二元樹，只要存每點的左右孩子編號就可以了。對於一棵有根樹的某個中間節點，其下的所有節點稱為子樹(subtree)，因此我們需要計算的其實是以每個點為根的子樹重量。計算子樹重量的方法我們在 P-8-3 算 median 的時候已經看過了，這一題的中間節點本身是沒有重量，所以是計算子樹中的葉節點重量總和，但做法沒有不同。

我們來看看模擬的流程。對於每個中間點 v ，令 $w[v]$ 為其下貨箱總重， $LC[v]$ 與 $RC[v]$ 分別是他的左右子節點的編號。根據題目的定義，中間節點的編號皆小於 n ，因此我們的流程可以描述如下：

```
// 進入貨物的重量為 x，計算其最後到達的貨箱
v = 1; // 進入點(root)的編號
while (v < n) // 尚未到達貨箱
    if w[LC[v]] <= w[RC[v]] then
        v = LC[v]; // 左邊較輕往左流
    else
        v = RC[v]; // 往右流
    end if;
    w[v] = w[v] + x; // 更新總重
end while;
輸出 v; // 到達的貨箱編號
```

下面的範例程式中，第 5~9 行的副程式 `find_w()` 是以 DFS 寫法來計算 v 點以下葉節點的總重，因為是二元樹，所以遞迴呼叫左右孩子就可以，重點是要用表格紀錄。主程式在輸入資料後，第 24 行從 `root` 呼叫計算重量會把所有節點的重量全部算完，然後第 26 行開始模擬貨物進來的流程。


```

00 // P-8-5 arbitrary binary tree, DFS DP
01 #include <stdio.h>
02 #define N 200010
03 int n, w[N]={0}, lc[N], rc[N];
04 // recursively compute the weight
05 int find_w(int v) {
06     if (v>=n) return w[v]; // leaf
07     w[v] = find_w(lc[v]) + find_w(rc[v]);
08     return w[v];
09 }
10
11 int main() {
12     int m, i, in[110];
13     scanf("%d%d", &n, &m);
14     for (i=n; i<n*2; i++)
15         scanf("%d", &w[i]);
16     for (i=0; i<m; i++)
17         scanf("%d", &in[i]);
18     for (i=0; i<n-1; i++) {
19         int v;
20         scanf("%d", &v);
21         scanf("%d%d", &lc[v], &rc[v]);
22     }
23     // find initial weight of internal node
24     w[1] = find_w(1); // DFS from root=1
25     // start simulation
26     for (i=0; i<m; i++) {
27         int v=1;
28         while (v<n) { // until reaching leaf
29             if (w[lc[v]] <= w[rc[v]])
30                 v = lc[v]; // go left
31             else v = rc[v]; // go right
32             w[v] += in[i];
33         }
34         if (i>0) printf(" "); // output space
35         printf("%d", v);
36     }
37     printf("\n");
38     return 0;
39 }

```

下面一題是一個裸題，要算樹上所有點到所有點的距離。 $O(n^3)$ 當然不能接受， $O(n^2)$ 也不行，需要一個 $O(n)$ 的算法。這一題當做習題，給一個提示：請看看 P-8-3 題中計算 median 到所有點的距離和所使用的方法。

Q-8-6. 樹狀圖的距離總和

輸入一個樹狀圖，請計算所有點到所有點的距離總和。本題假設編號 1 是根節點。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，代表點數，點以 $1 \sim n$ 編號，第二行有 $n-1$ 個正整數， $p(2), p(3), \dots, p(n)$ ，依序是編號 $2 \sim$ 編號 n 各點的 parent，第三行有 $n-1$ 個正整數，依序代表 i 從 2 到 n ，邊 $(i, p(i))$ 的長度。 n 不超過 $1e5$ ，每條邊長度是不超過 1000 的正整數。

輸出：各點到各點的距離總和。請注意 i 到 j 與 j 到 i 都要納入總和，如範例。

範例一輸入： <pre>4 1 2 3 10 20 30</pre>	範例一輸出： <pre>400</pre>
範例二輸入： <pre>5 1 1 1 2 20 20 30 30</pre>	範例二輸出： <pre>880</pre>

P-8-7. 寶石的顏色 (108 全國高中賽)

尋寶之旅的遊戲有一個地圖，地圖上有 n 個站，以 0 到 $n-1$ 編號，此外有 $n-1$ 條道路，這些道路都是單向的，遊戲固定從 0 號站出發，且已知從 0 號站出發可以直接或間接到達任何其他站。每一個站都有一顆寶石，寶石有分為多種顏色，第 i 站存放的寶石顏色為 $c(i)$ 。出發之前，你可以選定一種顏色的寶石收集箱，路途中遇到與你的收集箱相同顏色的寶石就可以收集 (包含起點)，請你計算最多可以收集到多少顆同色寶石。

Time limit: 1 秒

輸入格式：第一行有是一個正整數 n ， $n \leq 2e5$ ，代表地圖上有 n 個站。第二行是 n 個非負整數，依序代表每一站的寶石顏色號碼 $c(0), c(1), \dots, c(n-1)$ ；寶石的顏色號碼不超過 $1e9$ 。接著有 $n-1$ 行，每行兩個以空白間隔的整數 s 與 t ，表示有一條 s 到 t 的道路。

輸出：輸出為一整數，代表最多可能收集到的寶石數量。

範例一輸入： <pre> 6 0 0 0 0 0 0 0 1 1 2 0 3 1 4 1 5 </pre>	範例一輸出： <pre> 3 </pre>
範例二輸入： <pre> 10 5 3 3 1 4 0 3 4 5 0 0 1 5 2 7 3 5 4 0 5 4 6 1 7 0 8 4 9 </pre>	範例二輸出： <pre> 2 </pre>

(測資檔案中，第一筆測資只有一色，第二筆測資不超過 10 個顏色)

樹的每個點上有一個顏色，要選擇一個顏色，並且找出一條由根走到某個葉子的路徑，經過所選顏色的點數越多越好。先想比較簡單的例子，如果只有一種顏色，那就是找一個最長的路徑，因為只能由根往下走，也就是找出根的高度。在 P-8-4 我們做過如何計算樹的高度。把計算高度的程式稍微修改一下，只有同色的點才計算，很容易就可以計算一條路徑上某個顏色的點有多少，如下面的做法。

```

// max num of nodes of color col in a path
int dfs(int v, int col) {
    int nc = 0;
    for (int u: child[v]) {
        nc = max(nc, dfs(u,col));
    }
    return nc + (c[v]==col);
}

```

在多種顏色的狀況下，一個簡單的做法是把每一種顏色都算一算，但是時間複雜度會是 $O(Cn)$ ，其中 C 是顏色種類數量。同樣的路走那麼多次實在不聰明，我們可以準備一張表格，紀錄目前的路徑上，各種顏色的數量。DFS 過程中，每次走到一點，就將該點顏色數量加一；當 DFS 返回 parent 時，我們可以把該點顏色數減一，以表示回復沒有走到他的狀態。這樣看起來表格很容易維護，但是要如何找最多顏色呢？

如果我們在表格上找最大值，又需要花 $O(C)$ 的時間，如果我們聰明一點只有在葉節點才計算最大值，可能可以減少一點時間，但是葉節點的數量可能會高達 $O(n)$ ，所以還是不好。其實很簡單，在每個節點時，我們不必去看所有的顏色，因為只有該點的顏色數量有變動，所以只要檢查該點的顏色數量有沒有超過最大值就可以了。

最後還有一個問題，顏色的編號並非由 0 或 1 連續編號，而是 10 億以內的非負整數，這表格要如何做呢？其實我們前面已經看過了，這就是要做離散化 (P-2-2)。有多種方式可以做離散化，先將他們轉換成 $0 \sim C-1$ 也可以，下面的範例程式我們使用一個 map 當做顏色編號的字典，我們並不先去置換顏色編號，而是要用到時再去查詢編號。

```
// rooted tree, find a path with max same color
#include<bits/stdc++.h>
using namespace std;
#define N 200010
int n, ans=0, c[N]; // color of node
vector<int> child[N];
map<int,int> D; // a dictionary for id of color
int counter[N]={0}; // counter of each color

void dfs(int v) {
    int cc=D[c[v]]; // find id of color
    counter[cc]++; // add color of v
    ans = max(ans, counter[cc]); // check answer
    for (int u: child[v]) {
        dfs(u);
    }
    counter[cc]--; // recover
    return;
}

int main() {
    int i;
    scanf("%d", &n);
    for (i=0; i<n; i++) scanf("%d", &c[i]);
    for (i=0; i<n-1; i++) {
        int v, p;
        scanf("%d%d", &p, &v);
        child[p].push_back(v);
    }
    // discretization
    for (i=0; i<n; i++) D.insert({c[i], 0});
    int cnt=0; // id of color
```

```

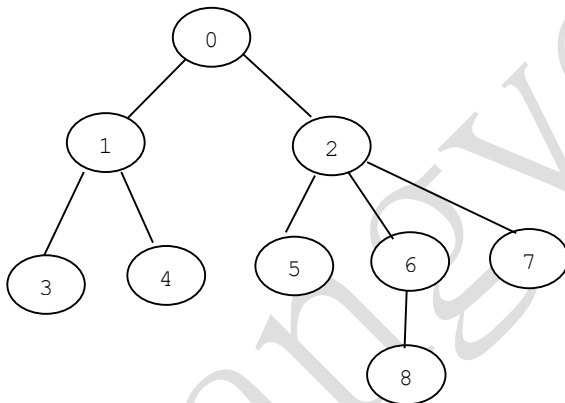
for (auto &v:D) {
    v.second=cnt;
    cnt++;
}
dfs(0); // dfs from root
printf("%d\n",ans);
return 0;
}

```

在圖論中，一群不相連的點稱為獨立集 (independent set)，在一般圖上找獨立集是件困難的事情，但是在樹上找獨立集並不困難。下面是一個相關的題目。

P-8-8. 樹的最大獨立集

輸入一棵有 n 個點的樹，我們要挑選一群彼此不相鄰的點，而且挑選的點越多點越好。請計算最多可以挑多少點。以下圖為例，我們可以挑 $\{0, 3, 4, 5, 6, 7\}$ 共 6 點，沒有辦法挑出更多的不相鄰的點。



Time limit: 1 秒

輸入格式：第一行是正整數 n ，代表點數，點以 $0 \sim n-1$ 編號， 0 是根，第二行有 $n-1$ 個正整數， $p(1), p(2), \dots, p(n-1)$ ，依序是編號 $1 \sim n-1$ 各點的 parent。 n 不超過 $1e5$ ，相鄰數字以空白間隔。

輸出：最多可以挑選幾點。

範例一輸入： 9 0 0 1 1 2 2 2 6	範例一輸出： 6
範例二輸入： 8 0 1 2 3 3 2 2	範例二輸出： 5

若 T 是我們要處理的樹，樹一定有葉，假設 v 是一個葉。我們宣稱：一定有一個最大獨立集挑選了 v 而沒有挑選 $\text{parent}(v)$ 。證明：假設某最大獨立集 S 沒有挑選 v ，那麼一定有挑 $\text{parent}(v)$ ，否則可以將 v 加入 S 中變成更大的獨立集。我們可以用 v 點取代 S 中的 $\text{parent}(v)$ 得到另外一個獨立集 S' ，就是我們宣稱包含 v 的最大獨立集。這樣的替換不可能造成不合法的解，因為 v 點是葉子， $\text{parent}(v)$ 是它唯一的鄰居。

根據以上的證明，我們可以得到一個貪心法：對於一棵樹，挑選所有的葉節點，捨棄這些葉節點的 parent ，重複此步驟直到樹被拔光為止。以題目中的樹為例，我們會先挑 $\{3, 4, 5, 8, 7\}$ ，捨棄 $\{1, 2, 6\}$ ；然後挑 0 ，結束。所得的獨立集是 $\{0, 3, 4, 5, 8, 7\}$ ，與題目中的解雖然不一樣，但都是最大獨立集。

實作時，可以一次拔掉全部的葉子，也可以一點一點拔，因為從葉子下手，所以我們需要一個由下而上的順序，當然也可以用 DFS 取代。以下是範例程式，程式的重點在於走到每一個點時，必須知道他的孩子中是否有人被挑選，我們用一個陣列 $\text{chosen}[]$ 紀錄每個點是否可以選，初始的時候全設為 1 ，當一個點被挑時，將它的 parent 的 chosen 設為 0 。這貪心法中我們沒有用表格紀錄而是用遞迴函數值接回傳結果，主程式中則只有讀取輸入與呼叫 $\text{dfs}(0)$ 而以。有一點要提醒， $\text{dfs}(r)$ 裡面會對 $\text{parent}[r]$ 做動作，但如果走到根節點時， $\text{parent}[r]$ 的存取有可能會造成傷害，所以我們 $\text{parent}[0]$ 設為不會出現的 n ，當然也有別種寫法。

```
// P-8-8 unweighted max independent set of a tree
// top-down implement
#include <bits/stdc++.h>
using namespace std;

#define N 100020
vector<int> child[N];
int parent[N], chosen[N]; // if chosen
// return max independent set, un-chosen its parent if chosen
int dfs(int r) {
    int num=0;
    for (int s:child[r]) {
```

```

        num += dfs(s);
    }
    if (chosen[r]) { // r is chosen
        chosen[parent[r]] = 0; // un-chosen its parent
        return num + 1;
    }
    return num; // r is not chosen
}

int main() {
    int i, n;
    scanf("%d", &n);
    for (i=1; i<n; i++) scanf("%d", &parent[i]);
    for (i=1; i<n; i++)
        child[parent[i]].push_back(i);
    for (i=0; i<n; i++) chosen[i]=1;
    parent[0] = n; // dummy
    printf("%d\n", dfs(0)); // root = 0
    return 0;
}

```

樹的貪心法用 bottom-up 其實比較自然，只是 bottom-up 的順序需要多一個 queue 來處理，以下是 bottom-up 的寫法。程式架構就是一個標準的由下而上的走訪，我們用了一個陣列 mark[] 來標記不可以挑選的點，當走到一點時，如果他沒有被標記，就挑選他並標記他的 parent。我們把 root 放在迴圈結束後處理，以免在迴圈內多寫 if。

```

// P-8-8 unweighted max independent set of a tree
// greedy, choosing leaf and mark its parent
// bottom-up implement
#include <bits/stdc++.h>
using namespace std;
#define N 100020
int parent[N], deg[N]={0};

int main() {
    int i, n, total=0;
    scanf("%d", &n);
    for (i=1; i<n; i++) {
        scanf("%d", &parent[i]);
        deg[parent[i]]++;
    }
    int mark[N]={0}; // cannot be chosen
    // bottom-up traversal
    queue<int> que;
    for (i=0; i<n; i++)
        if (deg[i]==0) que.push(i);
    i=0;
    while (! que.empty()) {
        int v=que.front();
        que.pop(); i++;
    }
}

```

```

    if (v==0) break;
    if (mark[v] == 0) {
        total++;
        mark[parent[v]] = 1;
    }
    if (--deg[parent[v]] == 0)
        que.push(parent[v]);
}
if (mark[0]==0) total++; // if root is chosen
printf("%d\n",total);
fprintf(stderr," visited nodes =%d\n",i);
return 0;
}

```

下面兩個問題都是與 P-8-8 類似的題目，我們當做習題。

Q-8-9. 服務中心選位置

有 n 個小村莊以 $1 \sim n$ 編號，這些村莊以 $n-1$ 條道路連接，每條道路都是雙向通行且連接兩個不同的村莊，已知從任兩個村莊之間都可以經由這些道路通達，兩個村莊稱為鄰近的村莊，如果它們之間有一條道路直接相連。現在要選一些村莊成立服務中心，因為預算的關係無法每個村莊都成立一個服務中心，政府的政策決定：如果某個村莊沒有設服務中心，那麼他一定有一個鄰近的村莊有服務中心。也就是說，任何一個村莊最多只要經過一條道路就可以到達一個有設服務中心的村莊。請你計算至少需要成立幾個服務中心。

Time limit: 1 秒

輸入格式：第一行是正整數 n ， $n > 1$ ，村莊以 $1 \sim n$ 編號。接下來有 $n-1$ 行是道路的資料，每一行有兩個正整數 u 與 v ，代表有一條道路連接 u 與 v 。 n 不超過 $1e5$ 。

輸出：最少需要成立的服務中心數量。

範例一輸入： <pre> 5 1 2 5 1 5 3 5 4 </pre>	範例一輸出： <pre> 2 </pre>
範例二輸入： <pre> 7 1 2 2 3 4 3 5 4 5 6 7 6 </pre>	範例二輸出： <pre> 3 </pre>

範例一說明：設在 $\{5, 2\}$ 或 $\{5, 1\}$ 皆符合要求。

範例二說明：這是 7 個點排成一條路徑，選 $\{2, 5, 7\}$ 是一個解，兩個點一定無法滿足需求。

Q-8-9 提示：我們說一個點可以管轄自己與它的鄰居，一個圖上選一些點可以管轄所有的點，這樣的集合稱為管轄集(dominating set)，本題要計算樹的最小管轄集。類似於 P-8-8，我們可以想一想：如果 v 是一個葉節點，是否一定有最佳解選了或沒選 v ？當決定選擇某點時，要將它鄰近的點做一些設定。

Q-8-9 是比較容易寫壞的題目，下面這一題比較容易。

Q-8-10. 竊聽間諜網

有 n 個間諜，他們的 ID 以 $0 \sim n-1$ 編號，間諜頭子的 ID 是 0，除了他以外每個間諜都有一個領導，每個間諜只會與他的領導相互通話，現在我們要竊聽間諜們彼此的通話，所以要在一些間諜的通話裝置上安裝竊聽器，要竊聽到某個間諜與他領導的通話，必須至少在他們其中一人的通話裝置上安裝竊聽器。如果不想漏掉任何通話，至少要安裝幾台竊聽器。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，代表間諜數，每個間諜的 ID 必定大於他領導的 ID。
第二行有 $n-1$ 個整數分別是 $t(1), t(2), \dots, t(n-1)$ ，其中 $t(i)$ 就是 i 的領導。 n 不超過 $1e5$ 。

輸出：最少的竊聽器數量。

範例一輸入： 5 0 1 1 3	範例一輸出： 2
範例二輸入： 7 0 1 1 1 1 1	範例二輸出： 1

範例一說明：安裝在 1 號與 3 號間諜。

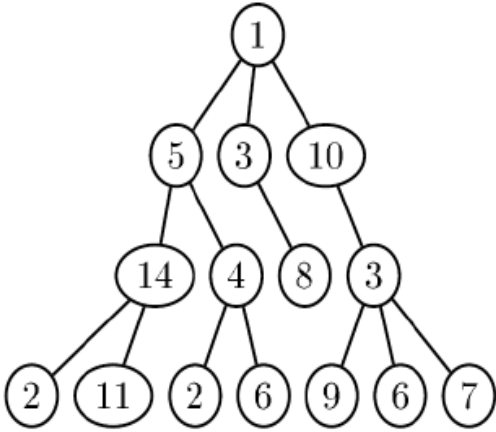
範例二說明：安裝在 1 號間諜。

我們說一個點可以 cover 它所接的邊，在一個圖中選一些點 cover 所有的邊，這樣的點集合稱為 vertex cover，也就是每一個邊的兩端點都至少被選中一點，本題是樹的無權重的 minimum vertex cover，與前兩題類似，所以不予提示。

P-8-11. 公司派對 (NCPC)

有一個公司有 n 個成員，成員以 $1 \sim n$ 編號，每個人都有一個領導，除了編號 1 是公司的總裁，他沒有領導。現在要辦一個公司派對，想要邀請一些人來參加，如果邀請了編號 i 的人來參加，就會有 $r(i)$ 的歡樂指數，可是，這個公司的任何人都不能跟他的領導一起參加派對，請你幫忙計算要邀請那些人來參加，才可以讓參加者的歡樂指數的總和最大。

右圖是一個例子，每個圓圈代表一個人，圓圈內的數字是他的歡樂指數，每個人上方與他連線的就是他的領導，你可以看到，總裁的歡樂



指數是 1，也全公司最低的。我們可以看到此例中有兩個人的歡樂指數是 14 與 11，可惜 14 是 11 的領導，所以不能同時邀請這兩人參加。這個例子的最大歡樂指數總和是 66，扣除歡樂指數分別為 1, 3, 3, 4, 14 那 5 個人，其餘的人都參加。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 與 $r(1)$ ，接下來有 $n-1$ 行，每行兩個正整數，由 $i=2$ 到 n ，依序是 $p(i)$ 與 $r(i)$ 。 n 不超過 $1e5$ ， $r(i)$ 不超過 1000 的正整數。

輸出：輸出參加派對的最大歡樂指數總和。

範例一輸入： <pre> 4 7 1 5 2 6 3 8 </pre>	範例一輸出： <pre> 15 </pre>
範例二輸入： <pre> 15 1 1 5 1 3 1 10 2 14 2 4 3 8 4 3 5 2 5 11 6 2 6 6 8 9 8 6 8 7 </pre>	範例二輸出： <pre> 66 </pre>

範例一說明：選編號 1 與 4，歡樂指數為 $7+8=15$ 。

範例二說明：題目敘述中的例子。

這一題其實是 P-8-8 的加權版本，是最大權重獨立集的問題。無加權時要挑選最多不相鄰的點，加權時要挑選總和最大的不相鄰的點。無加權時，貪心法則可以找到最多的點，但是在有加權時，貪心的方法並無法找到最佳解，我們嘗試用 DP 的觀點來找答案。對任一點 v ，令 $f(v)$ 代表以 v 為根的子樹的最佳解，也就是在 v 的後代(含 v)中，不相鄰的點最大可能的權重和，但在寫狀態轉移式時候遇到困難，因為 v 點是否有選會影響

響他的孩子是否可以挑選。碰到類似狀況時，把解分類是通常的解決方法，我們把解分成兩類：

$f_0(v)$ 是以 v 為根的子樹中，不挑選 v 的狀態下之最佳解；

$f_1(v)$ 是以 v 為根的子樹中，挑選 v 的狀態下之最佳解。

我們可以得到以下遞迴式：

$f_0(v) = \sum(\max(f_0(u), f_1(u)) : \text{for all children } u \text{ of } v);$

$f_1(v) = r(v) + \sum(f_0(u) : \text{for all children } u \text{ of } v);$

也就是說，當 v 不選的時候，他的孩子可以挑也可以不挑；而當 v 被挑選時，他的孩子都不可以挑。根據這個遞迴式，搭配由下而上的 DP 順序，就可以很容易求解，最後的答案在 $\max(f_0(r), f_1(r))$ ，其中 r 是根。實作時一樣有 bottom-up 與 top-down 兩種寫法，以下先揭示非遞迴的寫法，走訪的方式前面已經看過很多次了，就不再多解釋說明，時間複雜度是 $O(n)$ 。

```
// Tree, Max weight independent set, bottom-up DP
#include <bits/stdc++.h>
using namespace std;
#define N 100010
// f0[], f1[]: opt without/with root
// f0[r]=sum(max(f0[v],f1[v])), f1[r]=w[r]+sum(f0[v])
int main() {
    int parent[N], n, f0[N]={0}, f1[N], deg[N]={0};
    scanf("%d%d", &n, &f1[1]);
    for (int i=2; i<=n; i++) {
        scanf("%d%d", &parent[i], &f1[i]); // weight[i] in f1[i]
        deg[parent[i]]++;
    }
    queue<int> Q;
    for (int i=1; i<=n; i++)
        if (deg[i]==0) Q.push(i);
    while (!Q.empty()) {
        int v=Q.front();
        Q.pop();
        if (v==1) break;
        int r=parent[v];
        f0[r] += max(f0[v], f1[v]);
        f1[r] += f0[v];
        if (--deg[r] == 0) Q.push(r);
    }
    printf("%d\n", max(f0[1], f1[1]));
    return 0;
}
```

使用 top-down 的 DFS 時，輸入時先將各點的 child 設定好，這題有給 root，所以不必特別去找。程式的架構之前也看過多次，也不多做解釋了。

```
// Tree, Max weight independent set, top-down DP
#include <bits/stdc++.h>
using namespace std;
#define N 100010
int f0[N], f1[N], w[N];
vector<int> child[N];

void dfs(int r) {
    f0[r]=0;
    f1[r]=w[r];
    for (int c:child[r]) {
        dfs(c);
        f0[r]+=max(f0[c], f1[c]);
        f1[r]+=f0[c];
    }
    return;
}

int main() {
    int parent, n;
    scanf("%d%d", &n, &w[1]);
    for (int i=1; i<=n; i++) child[i].clear();
    for (int i=2; i<=n; i++) {
        scanf("%d%d", &parent, &w[i]);
        child[parent].push_back(i);
    }
    dfs(1);
    printf("%d\n", max(f0[1], f1[1]));
    return 0;
}
```

回顧一下在第六章的 P-6-2 (不連續的表演酬勞)，其實上那就是一個一維陣列的最大獨立集問題，而一維陣列是樹的一種特殊形式，所以也可以仿效 P-6-2 的解法來想這一題的 DP。如果用一維陣列時的解法，還有一種寫法是只設一個遞迴函數 $f(v)$ 是 v 子樹的最佳解， v 不挑時，選各子樹的解相加；而 v 挑選時，將 v 的孫子 (孩子的孩子) 為根的子樹的解拿來相加。因為一個點只有一個祖父，這樣做的複雜度還是 $O(n)$ ，所附範例程式中有一支是用這個方法寫的，請自行參考。

下一題在一維陣列中也有類似的題目，當做習題，提示是：請參考 P-4-13。

Q-8-12. 佔領連續的城鎮

某個遊戲中有 n 個城鎮以 $1 \sim n$ 編號，這些城鎮以 $n-1$ 條道路連接，每條道路都是雙向通行且連接兩個不同的城鎮，已知從任兩個城鎮之間都可以經由這些道路通達。現

在你要佔領一些城鎮來獲取最大的利益，佔領編號 i 的城鎮可以獲得 $w[i]$ 的利益，但是你佔領的城鎮必須內部可以互相通達，也就是說，若 i 與 j 都是你的領地，則從 i 出發可以只經過你自己的城鎮到達 j 。請計算你最多可以獲得多少總和利益。

Time limit: 1 秒

輸入格式：第一行是正整數 n ， $n > 1$ ，村莊以 $1 \sim n$ 編號。第二行有 n 個整數，依序是 $w[1], w[2], \dots, w[n]$ 。接下來有 $n-1$ 行是道路的資料，每一行有兩個正整數 u 與 v ，代表有一條道路連接 u 與 v 。 n 不超過 $1e5$ ，每個城鎮利益的絕對值不超過 $1e4$ 。

輸出：最多可以獲得的總和利益。

範例一輸入： 5 -2 3 -1 3 -4 1 2 5 1 5 3 5 4	範例一輸出： 3
範例二輸入： 7 2 -1 4 -3 5 -3 2 1 2 2 3 4 3 5 4 5 6 7 6	範例二輸出： 7

範例一說明：選 $\{2\}$ 或 $\{4\}$ 都是 3。

範例二說明：這是 7 個點排成一條路徑，選 $\{1, 2, 3, 4, 5\}$ ，總和是 7。

下面一題看起來也是類似的題目，他是 Q-8-9 的加權版本，但這一題比較困難。

P-8-13. 不同成本的購物中心 (@@)

有 n 個小村鎮以 $1 \sim n$ 編號，這些村鎮以 $n-1$ 條道路連接，每條道路都是雙向通行且連接兩個不同的村鎮，已知從任兩個村莊之間都可以經由這些道路通達，兩個村莊稱

為鄰近的村莊，如果它們之間有一條道路直接相連。現在要選一些村莊成立服務中心，因為預算的關係無法每個村莊都成立一個服務中心，政府的政策決定：如果某個村莊沒有設服務中心，那麼他一定有一個鄰近的村莊有服務中心。也就是說，任何一個村莊最多只要經過一條道路就可以到達一個有設服務中心的村莊。每個村莊設立服務中心的成本不同，在編號 i 的村莊成立服務中心需要的成本是 $w[i]$ ，我們希望以最少的總成本設置服務中心，請你計算最少的總成本。

Time limit: 1 秒

輸入格式：第一行是正整數 n ， $n > 1$ ，村莊以 $1 \sim n$ 編號。第二行有 n 個正整數，依序是 $w[1], w[2], \dots, w[n]$ 。接下來有 $n-1$ 行是道路的資料，每一行有兩個正整數 u 與 v ，代表有一條道路連接 u 與 v 。 n 不超過 $1e5$ ，每個村莊設置成本不超過 1000 。

輸出：需要成立服務中心的最少總成本。

範例一輸入： 5 2 4 1 3 7 1 2 5 1 5 3 5 4	範例一輸出： 6
範例二輸入： 7 1 3 3 1 2 2 1 1 2 2 3 4 3 5 4 5 6 7 6	範例二輸出： 3

範例一說明：設在 $\{1, 3, 4\}$ 成本 6。

範例二說明：這是 7 個點排成一條路徑，選 $\{1, 4, 7\}$ 成本是 3。

這一題是要計算 minimum weight domination set，與最大獨立集類似的地方是：無加權時可以貪心，加權時必須用 DP 來解。最大獨立集的 DP 遞迴式比較容易，這一

題的困難點並不是程式難寫或是需要什麼特別的資料結構與演算法，而是 DP 遞迴式很容易弄錯。子問題都定義在以任一點 r 為根的子樹的解，我們將子問題分成三類：

- $d1(r)$ ：在 r 被挑選下的最佳解 (domination set 的最小權重)。
- $d01(r)$ ：在 r 不被挑選但 r 已被管轄下的最佳解，也就是至少有一個 r 的孩子被挑選。
- $d00(r)$ ：在 r 不被挑選且 r 不一定有沒有被管轄之下的最佳解。這個值未必是個合法的解，根 r 要靠上面的點管轄。

子問題的遞迴式如下 (v 皆為對所有 r 的孩子)：

- $d1(r) = w(r) + \sum(\min(d00(v), d1(v)) : \text{all } v \text{ in child}(r))$ 。也就是對所有子樹的最小值加總後加上 r 的權重，因為 $d00(v) \leq d01(v)$ ，所以只需要取 $\min(d00(v), d1(v))$ 。
- $d00(r) = \sum(\min(d1(v), d01(v)) : \text{all } v)$
- $d01(r) = \sum(\min(d1(v), d01(v)) : \text{all } v) + \min_diff$ ；其中 $\min_diff$ 代表 $\max(0, \min(d1(v) - d01(v) : \text{all } v))$ ，也就是對所有孩子 v 找出 $d1(v) - d01(v)$ 的最小值，這個值是正的就要加上，如果是負的就不加。遞迴式的意義在各孩子的最小合法解之總和，但是不能全部的孩子都沒有挑選。

程式並不特別難，一樣有 top-down 與 bottom-up 兩種寫法，以下是 top-down 的寫法，註解快要比程式碼還多了，另外一種寫法類似，請自行參考。

```
// P-8-13 min domination set of a tree
// top-down DP d1(r): min cost with root,
// d01(r): without root but root is dominated
// d00(r): without root and root is not necessarily dominated
// d1(r)=w(r)+sum(min(d00(v),d1(v)): all child v of r), d00<=d01
// d00(r)=sum(min(d1(v),d01(v)))
// d01(r)=sum(min(d1(v),d01(v)))+ max(0,min(d1[v]-d01(v)))
// not all d01
#include <bits/stdc++.h>
using namespace std;

#define N 100020
#define oo 1000000000
vector<int> adj[N];
int w[N], d1[N], d01[N], d00[N];
// set min domination set of subtree(r) with parent p
void dfs(int r, int p) {
    d1[r] = w[r], d01[r]=oo, d00[r]=0;
    for (int v: adj[r]) {
        if (v==p) continue;
```



```

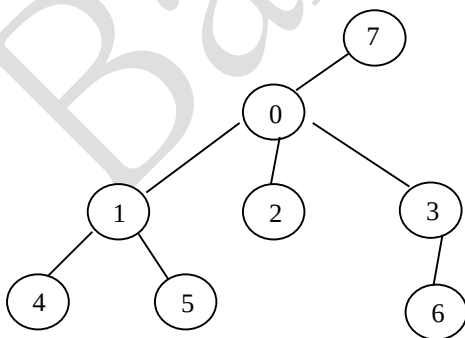
    dfs(v,r);
    d1[r] += min(d00[v], d1[v]);
    d00[r] += min(d1[v], d01[v]);
    d01[r] = min(d01[r], d1[v]-d01[v]);
}
d01[r] = d00[r] + max(0, d01[r]);
return;
}

int main() {
    int i, n;
    scanf("%d",&n);
    for (i=1; i<=n; i++) scanf("%d",&w[i]);
    for (i=1; i<n; i++) {
        int u,v;
        if (scanf("%d%d",&u, &v)!=2)
            fprintf(stderr,"input error\n");
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    // parent of root(1) is set to 0(dummy);
    dfs(1,0);
    printf("%d\n", min(d1[1], d01[1]));
    return 0;
}

```

P-8-14. 血緣關係 (APCS201603)

小宇有一個大家族。有一天，他發現記錄整個家族成員和成員間血緣關係的家族族譜。小宇對於最遠的血緣關係（我們稱之為"血緣距離"）有多遠感到很好奇。



上圖為家族的關係圖。0 是 7 的孩子，1、2 和 3 是 0 的孩子，4 和 5 是 1 的孩子，6 是 3 的孩子。我們可以輕易的發現最遠的親戚關係為 4 (或 5) 和 6，他們的"血緣距離"是 4 (4~1, 1~0, 0~3, 3~6)。我們將給你小宇家族的關係圖，請幫小宇找

出最遠的"血緣距離"。你可以假設只有一個人是整個家族成員的祖先，而且沒有兩個成員有同樣的小孩。

Time limit: 1 秒

輸入格式：第一行為一個正整數 n 代表成員的個數，每人以 $0 \sim n-1$ 之間惟一的編號代表。接著的 $n-1$ 行，每行有兩個以一個空白隔開的整數 a 與 b ($0 \leq a, b \leq n-1$)，代表 b 是 a 的孩子。 n 不超過 $1e5$ 。

輸出：輸出最遠"血緣距離"的答案。

範例一輸入： <pre> 8 0 1 0 2 0 3 7 0 1 4 1 5 3 6 </pre>	範例一輸出： <pre> 4 </pre>
範例二輸入： <pre> 4 0 1 0 2 2 3 </pre>	範例二輸出： <pre> 3 </pre>

範例一是題目中敘述的例子。

在一個圖中，最長的兩點最短路徑(距離)稱為直徑，直徑這個詞同時代表該路徑，也代表該路徑的長度。這個定義有點繞口，其實就像在平面幾何的圓中，直徑是圓上最遠的兩點的距離。如果對任意兩點 u, v 都求出兩點之間的最短距離 $d(u, v)$ ，那麼 $\text{diameter} = \max(d(u, v) : \text{all } u, v \text{ in } V)$ 。我們在前面的題目中已經知道，樹上求一點與其它點的距離可以在 $O(n)$ 時間內完成，那麼直接依照定義求出所有點到所有點的距離然後計算直徑可以在 $O(n^2)$ 時間完成。我們需要一個更有效率的解法。

這一題是無加權樹的直徑，算是樹的簡單 DP 中的經典題。先看用標準 top-down DP 的寫法。因為直徑必然由某個葉節點往上走到某個點 r 之後，再從 r 的另外一個孩子一路往下走，退化的狀態是 r 只有一個孩子，那就只走到 r 為止，我們稱這樣的路徑為以 r 為最高點的路徑。也可以換句話說，直徑一定是以某個 r 為最高點的最長路徑。我們

只要對每個 r ，計算出以 r 為最高點的最長路徑，也就是 r 往下的兩條最長路徑，然後對所有的 r 取最大值就可以了，但是要小心的是，這兩條路徑必須經由兩個不同的孩子。我們前面 P-8-4 學過樹中點的高度，事實上我們要找的就是 r 的兩個高度最高的孩子，把他們的高度相加再加 2 就是以 r 為最高點的最長路徑。

以 top-down 寫法的主架構當然還是 DFS，在對所有孩子遞迴呼叫後，我們的任務是找出深度最高的兩個孩子，這就像在一個數列中找出最大的兩個值一樣。如果用直接的寫法，通常需要用兩次迴圈，第一次找出最大值，第二次找出非最大值中的最大值；或者掃描一次但同時維護好第一名與第二名。這兩種寫法的時間複雜度是 $O(n)$ ，因為對每一個點 r ，它需要的時間複雜度是 $O(|\text{child}(r)|)$ ，而所有點的孩子數量總和不過是 $O(n)$ 。但不管如何，這兩種寫法都有點囉嗦。想想下面這招：假設數列是 S ， pm 是 prefix_max ，則最大兩個元素之和其實就是 $\max(S[i] + \text{pm}(i) : \text{for all } i)$ 。

所以我們只需要一路掃描並維護好目前看到的最大值，每次將當下看到的值加上他之前的最大值相加即可。以下是範例程式，非常的簡潔漂亮。請留意，在 prefix_max 預設為 0 下，這個方法也適用只有一個孩子的情形。

```
// unweighted tree diameter, top-down DP
#include <bits/stdc++.h>
using namespace std;
#define N 100010
vector<int> child[N]; // list of children
int hei[N]={0}, diameter=0;
// find longest path whose top-vertex is r
void dfs(int r) {
    for (int v: child[r]) {
        dfs(v);
        diameter = max(diameter, hei[v]+1+hei[r]);
        // update parent's height
        hei[r] = max(hei[r], hei[v]+1);
    }
}

int main() {
    int i,j,k,n;
    bool not_root[N]={false};
    scanf("%d", &n);
    for (i=0; i<n-1; i++) {
        scanf("%d%d", &j, &k);
        child[j].push_back(k);
        not_root[k] = true;
    }
    // find root
    int root=0;
    while (not_root[root]) root++;
    // dfs to find diameter
```

```

dfs(root);
printf("%d\n", diameter);
return 0;
}

```

上面的 DP 當然也可以寫成非遞迴的 bottom-up 形式，這已經出現很多次，就不再揭示這樣的寫法。樹的直徑有另外一個演算法，但必須根據圖論的一個性質：「距離樹上任一點最遠的點，一定是某個直徑的端點」。這個性質的證明並不算難，但我們只簡要說明，有興趣的讀者自己去證明。若 v 是任一點， u 是離 v 最遠的一點，而 D 是任一條直徑，則 v 到 u 的路徑與 D 必然相交；再則， u 到 D 的兩端較遠者，必然也是直徑。

根據上述的性質，可以得到一個很簡單的線性時間演算法，我們稱為 farthest of farthest 演算法：挑選任一點 v ，找離它最遠的一點 u ，再找離 u 最遠的一點 s ，則 u 到 s 就是直徑。以下是這樣的範例程式，我們以無根樹的方式用 BFS 來找距離最遠的點。因為要呼叫兩次，把 BFS 寫成函數，每次呼叫會算出起點到所有點的距離放在 $d[]$ 中，根據 $d[]$ 可以找出最遠的點與最遠的距離。

```

// unweighted tree diameter, farthest of farthest algorithm
// using bfs
#include <bits/stdc++.h>
using namespace std;
#define N 100010
vector<int> adj[N]; // adjacency list
int n, d[N];
// find all distances d[v] from r
void bfs(int r) {
    for (int i=0; i<n; i++) d[i]=-1; // unvisited
    queue<int> Q;
    Q.push(r);
    d[r] = 0;
    while (!Q.empty()) {
        int v=Q.front();
        Q.pop();
        for (int u: adj[v]) {
            if (d[u] >= 0) continue; // visited
            d[u] = d[v] + 1;
            Q.push(u);
        }
    }
    return;
}

int main() {
    scanf("%d", &n);
    for (int i=0; i<n-1; i++) {
        int j, k;
        scanf("%d%d", &j, &k);
        adj[j].push_back(k);
    }
}

```

```
adj[k].push_back(j);
}
bfs(rand()%n); // random root
int farthest = max_element(d, d+n) - d;
bfs(farthest);
printf("%d\n", *max_element(d, d+n));
return 0;
}
```

下面一題是與直徑有關的習題，提示是：想想 P-8-1 的解答說明。此外，這一題是加權版，也就是邊是有不同長度的。

Q-8-15. 樹上一位不回家的推銷員

有一個推銷員要走訪 n 個城市。這些城市以 $n-1$ 條道路連接，每條道路連接兩個不同的城市並且雙向可以通行，而且已知每一個城市都可以到達，不會有到達不了的狀況。輸入道路的資料，請你幫推銷員找出一個最短的路徑走訪所有的城市，推銷員可以從任何城市開始，也不必回到開始的城市，只要每個城市至少到一次就可以。

舉例來說，如右圖，有 5 個城市以 4 條道路連接，假設每條道路的長度都是 1。最短的拜訪路徑是 $(1, 0, 4, 2, 4, 3)$ ，長度為 5。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，代表城市的數量，城市是以 $0 \sim n-1$ 編號，接下來有 $n-1$ 行是道路的資料，每行三個整數 a, b 與 w ，代表此道路連接城市 a 與 b ，道路的長度是 w 。 n 不超過 50000，每條道路長度是不超過 100 的正整數。

輸出：輸出最短的旅行距離。

範例一輸入：

5
1 0 1
0 4 1
2 4 1
4 3 1

範例一輸出：

5

範例二輸入：

3
1 2 5
0 2 3

範例二輸出：

8

--	--

Q-8-16. 病毒演化 (APCS202007)

在資訊科學上，RNA 病毒可以看成一個由 A、U、G、C 這四種字母構成的字串。病毒演化中，某些字元可能變異成其他字元，例如將 AAUGG 中的第 3 個位置的 U 換成 C，則得到 AACGG，變異可能發生在多個位置。有個實驗團隊取得了某種 RNA 病毒的數個 RNA 片段樣本，並且掌握到這些樣本演化的親緣關係，用

(樣本編號，親代編號，RNA 字串) 的格式記錄，其中「樣本編號」與「親代編號」為兩個整數，若兩數字相同，則該樣本即為演化的源頭。例如，

(1, 1, AAAA)

(2, 1, GCAA)

表示樣本 1 的 RNA 字串為 AAAA，樣本 2 的為 GCAA，且樣本 2 是由樣本 1 在兩個位置發生變異演化而來，樣本 1 為演化的源頭。

然而，字串中每個位置的字元無法確定，實驗團隊以 @ 來表示這些字元，換言之，@ 可能為 A、U、G、C 中任何一個。請注意，一個字串中可能有多個 @，並非代表這些位置的字元是相同的。例如 A@C@ 可能是任何第一個字元為 A 且第三個字元為 C 的字串，像是 ACCU 或 AACC。團隊猜測演化過程發生的變異總數會盡可能的少。請你利用親緣關係，來計算最小的變異總數量。

Time limit: 1 秒

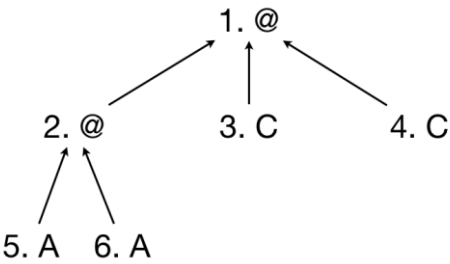
輸入格式：第一行有兩個正整數 n 與 m ，表示共有 n 個樣本，由 1 至 n 編號；每個樣本之 RNA 字串長度均為 m ，其中 $n \leq 1000$ 且 $m \leq 80$ 。接下來 n 行，每行包含以空白間隔的兩個正整數 i 與 j 以及一個 RNA 字串 s_i ，對應一個樣本 (i, j, s_i) ， s_i 由 A、U、G、C 與 @ 五種字元所組成。若 $i=j=1$ ，則該樣本即為演化的源頭，源頭以外的樣本皆恰有一個親代。

輸出：輸出最小可能的變異總數。

範例一輸入： 2 3 1 1 AAC 2 1 A@@	範例一輸出： 0
範例二輸入： 6 1 1 1 @ 2 1 @ 3 1 C 4 1 C 5 2 A 6 2 A	範例二輸出： 1

範例一說明：樣本 2 為 AAC 時，變異量為 0。

範例二說明：我們以一樹狀結構表示此例的親緣關係：



將樣本 1 置換成 C、樣本 2 置換成 A，則只有在樣本 1 與 2 之間有一個變異，其它皆無變異，故答案是 1。

Q-8-16 提示：從範例二看到的第一件事是這些 RNA 是個樹狀結構，另外一個很重要的事情是：雖然每個點是個字串，但是字串的每個位置是獨立的，也就是所有字串的第 1 個字元，要算一個最小變異量；所有字串的第 2 個字元，要算一個最小變異量；對每一個字串位置 i 都要算一個最小變異量，最後的答案是把他們加起來。所以，我們只要會解字串長度 $m=1$ 的狀況，最後套一個迴圈就好了。

如果沒有 '@'，所有的變異都是確定的，只要把每個點與它的 parent 比較是否相等就好了，所以問題在如何決定 '@' 變異量。如果葉節點是 '@'，我們可以讓它跟它的 parent 相同，這顯然不會有變異量，也就是說 '@' 的葉子根本可以無視它。為了計算方便，我們先將 AUCG 轉換為數字 0~3，以 DP 的觀點，對於每個節點 v ，我們算四個成本 $\text{cost}[v][i]$ ， i 從 0 到 3，分別代表如果 parent 的字元是 i 的時候 v 點以下子樹的總成本，然後去建構出遞迴式。

Bangye.Wu